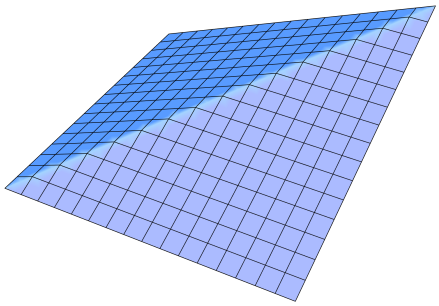

CO 372: Portfolio Optimization Models

Prof. Wei Xu • Fall 2014 • University of Waterloo
TRF 8:30–9:50AM in MC 1085

Transcribed, LaTeXed, and edited by Rob Zimmerman

Note: These are personal lecture notes, not official course materials. They may contain errors (which by default you should attribute to me, Rob Zimmerman, rather than the course instructor). The permanent home of these — and all of my other lecture notes — is https://rob-zimmerman.github.io/coursenotes.

Contents

1. Applied Linear Algebra

Lecture 1 — Tuesday, September 09, 2014

Course Overview and Motivation

This course studies the computational optimization methodology underlying portfolio problems. The material is organized into three connected parts:

1. applied numerical linear algebra,
2. computational optimization, and
3. portfolio optimization models.

The first part develops **numerical linear algebra** tools that we will need later. The core topics include:

1. solving square linear systems,
2. handling underdetermined and overdetermined systems,
3. LU and QR factorizations, and
4. eigenvalue decomposition and singular value decomposition.

These methods provide the computational foundation for solving real optimization models efficiently and accurately.

The second part turns to **nonlinear optimization**. Since derivatives are central in nonlinear problems, we study several ways to compute them:

1. finite difference methods,
2. symbolic computation, and
3. automatic differentiation.

We then study Newton's method for nonlinear equations and optimization, together with direction search ideas and the trust region method for improving stability. Finally, we introduce constrained

optimization algorithms, with special attention to constrained quadratic problems because of their strong connection to portfolio models.

After developing linear and nonlinear solvers, we apply them to **portfolio optimization** problems in finance. A central example is the Markowitz portfolio model, which balances expected return against risk and leads naturally to quadratic programming formulations. We also study the capital asset pricing model (CAPM), which gives a theoretical expected return for a risky asset, and the Sharpe ratio, which measures excess return per unit of risk.

The portfolio portion of the course is organized around three questions:

1. How do we choose an optimal portfolio with maximum expected return or minimum risk?
2. Given the market risk, what is the appropriate expected return for an individual risky asset?
3. If two risky portfolios are available, which one is the better investment?

MATLAB is the main computational environment for the course. It is especially convenient for matrix manipulation and also provides optimization and finance functionality that we will use throughout the term.

Lecture 2 — Thursday, September 11, 2014

Suppose that we want to invest in risky assets $S_1, \ldots, S_n$. The **efficient frontier** records the portfolios whose weights balance risk and return optimally.

The **capital asset pricing model** (**CAPM**) gives the theoretical relation for the expected return on the ith asset as

$$\mathbb{E}[R_i] = r_f + \beta_i \left[\mathbb{E}[R_m] - r_f\right],$$

where r_f is the **risk-free return**, β_i is the ith asset's **beta**, R_m is the **market return**, and $\mathbb{E}[R_m] - r_f$ is the **market risk premium**.

The **Sharpe ratio** measures excess return per unit of risk:

$$S_a = \frac{\mathbb{E}[R_a - r_b]}{\sigma_a}.$$

Here, R_a is the return of asset a, and r_b is the benchmark return (often the risk-free or market

return). Also,

$$\sigma_a = \sqrt{\mathrm{Var}(R_a - r_b)}.$$

Gaussian Elimination and LU Factorizations

Let $\mathbf{A} \in \mathbb{R}^{n\times n}$ be nonsingular, and let $\boldsymbol{b} \in \mathbb{R}^n$. We want to solve the system

$$\mathbf{A}\boldsymbol{x} = \boldsymbol{b}.$$

Although the solution is formally $\boldsymbol{x} = \mathbf{A}^{-1}\boldsymbol{b}$, explicitly computing $\mathbf{A}^{-1}$ is computationally expensive and numerically unattractive. What else can we do?

Gaussian elimination has two parts:

1. Use elementary row operations to transform $\mathbf{A}$ into an upper triangular matrix. In the first column, for example, set $m_{i1} = a_{i1}/a_{11}$ for $i \geq 2$ and replace row i by row i minus m_{i1} times row 1.
2. Solve the resulting upper triangular system by back substitution.

Example 2.1 Consider

$$\begin{cases} x_1 + 2x_2 - 3x_3 = 3, \\ 2x_1 - 3x_2 + x_3 = 1, \\ -x_1 + 8x_2 + 2x_3 = 0. \end{cases}$$

The first column multipliers are $m_{21} = 2$ and $m_{31} = -1$. After eliminating the first column and then the second column, we obtain

$$\begin{cases} x_1 + 2x_2 - 3x_3 = 3, \\ -7x_2 + 7x_3 = -5, \\ 9x_3 = -29/7. \end{cases}$$

Back substitution gives

$$x_3 = -\frac{29}{63}, \qquad x_2 = \frac{16}{63}, \qquad \text{and} \qquad x_1 = \frac{10}{9}.$$

The pivot a_{11} cannot be zero. If it is zero, or even very small, we interchange rows before eliminating. More generally, **partial pivoting** chooses a pivot with large absolute value, usually the

largest available entry in the current column, and moves it into position by interchanging rows.

Example 2.2 If we wish to solve

$$\begin{cases} 0x_1 + 3x_2 + 2x_3 = 1 \\ 3x_1 + 3x_2 + x_3 = 0 \\ x_1 + 2x_2 + 3x_3 = 0 \end{cases},$$

we can start by interchanging the first two rows to obtain the equivalent system

$$\begin{cases} 3x_1 + 3x_2 + x_3 = 0 \\ 0x_1 + 3x_2 + 2x_3 = 1 \\ x_1 + 2x_2 + 3x_3 = 0 \end{cases}$$

and solve as usual. This row interchange creates a nonzero first pivot, after which Gaussian elimination proceeds as usual.

In MATLAB, the preferred command is `x = A\b`, which performs Gaussian elimination. The command `x = inv(A)*b` is mathematically meaningful, but it should not be used for numerical linear solves because it explicitly forms the inverse.

Gaussian elimination can be written in matrix form as an LU factorization. In its simplest form,

$$\mathbf{A} = \mathbf{LU},$$

where $\mathbf{L}$ is unit lower triangular and $\mathbf{U}$ is upper triangular. To solve $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$, factor $\mathbf{A} = \mathbf{LU}$, solve $\mathbf{L}\boldsymbol{y} = \boldsymbol{b}$, and then solve $\mathbf{U}\boldsymbol{x} = \boldsymbol{y}$.

Lecture 3 — Friday, September 12, 2014

Example 3.1 For another pivoting example, let $\mathbf{A} = \begin{pmatrix} 1 & 1 & 3 \\ 2 & 1 & 2 \\ 1 & -1 & -1 \end{pmatrix}$ and $\boldsymbol{b} = \begin{pmatrix} 1 \\ 0 \\ 5 \end{pmatrix}$, and solve $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$.

$$\begin{cases} x_1 + x_2 + 3x_3 = 1 \\ 2x_1 + x_2 + 2x_3 = 0 \\ x_1 - x_2 - x_3 = 5 \end{cases} \xrightarrow{\text{pivot}} \begin{cases} 2x_1 + x_2 + 2x_3 = 0 \\ x_1 + x_2 + 3x_3 = 1 \\ x_1 - x_2 - x_3 = 5 \end{cases} \xrightarrow[m_{31}=-1/2]{m_{21}=-1/2} \begin{cases} 2x_1 + x_2 + 2x_3 = 0 \\ 0 + \frac{1}{2}x_2 + 2x_3 = 1 \\ 0 - \frac{3}{2}x_2 - 2x_3 = 5 \end{cases}$$

Continuing the elimination gives $x_3 = 2$, $x_2 = -6$, and $x_1 = 1$.

When solving a system using Gaussian elimination, the cost of elimination is $(2/3)n^3 + O(n^2)$ flops, while backward substitution costs $O(n^2)$ flops. The total cost is therefore $(2/3)n^3 + O(n^2)$ flops, which is $O(n^3)$.

In MATLAB, once $\mathbf{A}$ and $\boldsymbol{b}$ have been entered, the command `x = A\b` solves the system. MATLAB warns when $\mathbf{A}$ is singular, nearly singular, or badly scaled, but it ordinarily still returns a result. Near-singularity is assessed through conditioning, not by comparing $\det(\mathbf{A})$ with a fixed threshold; the determinant is scale-dependent and therefore unsuitable for that test.

Another method is **LU factorization**, which is mathematically equivalent to Gaussian elimination:

1. Factor $\mathbf{PA} = \mathbf{LU}$, where $\mathbf{L}$ is unit lower triangular, $\mathbf{U}$ is upper triangular, and $\mathbf{P}$ is a permutation matrix.
2. Solve $\mathbf{L}\boldsymbol{y} = \mathbf{P}\boldsymbol{b}$.
3. Solve $\mathbf{U}\boldsymbol{x} = \boldsymbol{y}$.

In MATLAB, this is written as `[L,U,P] = lu(A)`, `y = L\(P*b)`, and `x = U\y`.

This approach stores $\mathbf{L}$, $\mathbf{U}$, and $\mathbf{P}$ explicitly, whereas `x = A\b` does not. The factorization step costs $O(n^3)$, but once the factors are available, each new right-hand side can be solved using only the two triangular solves, each costing $O(n^2)$.

We still want to solve $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$. A matrix $\mathbf{A}$ is symmetric if $\mathbf{A} = \mathbf{A}^\top$, and it is positive definite if $\boldsymbol{x}^\top \mathbf{A}\boldsymbol{x} > 0$ for all nonzero $\boldsymbol{x}$. Equivalently, a real symmetric matrix is positive definite if all of its eigenvalues are positive. If $\mathbf{A}$ is symmetric positive definite, then there is a nonsingular lower triangular matrix $\mathbf{L}$ such that

$$\mathbf{A} = \mathbf{L}\mathbf{L}^\top.$$

This is the **Cholesky factorization**.

Lecture 4 — Tuesday, September 16, 2014

For $\mathbf{A} \in \mathbb{R}^{n \times n}$ and $\boldsymbol{b} \in \mathbb{R}^n$, we have now seen several ways to solve $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$:

1. Gaussian elimination with partial pivoting.
2. MATLAB's linear solve command `x = A\b`.
3. LU factorization: factor $\mathbf{PA} = \mathbf{LU}$, solve $\mathbf{L}\boldsymbol{y} = \mathbf{P}\boldsymbol{b}$, and solve $\mathbf{U}\boldsymbol{x} = \boldsymbol{y}$.

If $\mathbf{A}$ is known ahead of time to be symmetric positive definite, then the Cholesky factorization gives a more efficient method:

1. Factor $\mathbf{A} = \mathbf{L}\mathbf{L}^\top$.
2. Solve $\mathbf{L}\boldsymbol{y} = \boldsymbol{b}$.
3. Solve $\mathbf{L}^\top\boldsymbol{x} = \boldsymbol{y}$.

The Cholesky factorization costs $(1/3)n^3 + O(n^2)$ flops, about half the leading-order cost of LU factorization.

In MATLAB, `x = A\b` detects symmetric positive definite structure when possible and uses Cholesky factorization. Alternatively, we may write `R = chol(A)`, `y = R'\b`, and `x = R\y`; MATLAB's default factor is upper triangular, so $\mathbf{A} = \mathbf{R}^\top\mathbf{R}$. If $\mathbf{A}$ is not positive definite, `chol(A)` fails, although numerical roundoff means that its behaviour on positive semidefinite matrices can vary. The two-output form `[R,flag] = chol(A)` reports failure through `flag` rather than raising an error, making it a fast test for positive definiteness.

Example 4.1 Let

$$\mathbf{A} = \begin{pmatrix} 4 & 2 & -2 \\ 2 & 2 & -3 \\ -2 & -3 & 14 \end{pmatrix} \quad \text{and} \quad \boldsymbol{b} = \begin{pmatrix} 1 \\ 0 \\ 1 \end{pmatrix}.$$

To solve $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$ by Cholesky, first find $\mathbf{L}$ such that $\mathbf{A} = \mathbf{L}\mathbf{L}^\top$. Here

$$\mathbf{L} = \begin{pmatrix} \ell_{11} & 0 & 0 \\ \ell_{21} & \ell_{22} & 0 \\ \ell_{31} & \ell_{32} & \ell_{33} \end{pmatrix} \quad \text{and} \quad \mathbf{L}\mathbf{L}^\top = \begin{pmatrix} \ell_{11}^2 & \ell_{11}\ell_{21} & \ell_{11}\ell_{31} \\ \ell_{11}\ell_{21} & \ell_{21}^2 + \ell_{22}^2 & \ell_{21}\ell_{31} + \ell_{22}\ell_{32} \\ \ell_{11}\ell_{31} & \ell_{21}\ell_{31} + \ell_{22}\ell_{32} & \ell_{31}^2 + \ell_{32}^2 + \ell_{33}^2 \end{pmatrix}.$$

Matching entries with $\mathbf{A}$ and using positive diagonal entries for Cholesky factors,

$$\ell_{11}^2 = 4 \Rightarrow \ell_{11} = 2, \qquad \ell_{11}\ell_{21} = 2 \Rightarrow \ell_{21} = 1, \qquad \text{and} \qquad \ell_{11}\ell_{31} = -2 \Rightarrow \ell_{31} = -1,$$

$$\ell_{21}^2 + \ell_{22}^2 = 2 \Rightarrow 1 + \ell_{22}^2 = 2 \Rightarrow \ell_{22} = 1,$$

$$\text{and} \qquad \ell_{21}\ell_{31} + \ell_{22}\ell_{32} = -3 \Rightarrow -1 + \ell_{32} = -3 \Rightarrow \ell_{32} = -2,$$

$$\ell_{31}^2 + \ell_{32}^2 + \ell_{33}^2 = 14 \Rightarrow 1 + 4 + \ell_{33}^2 = 14 \Rightarrow \ell_{33} = 3.$$

Therefore,

$$\mathbf{L} = \begin{pmatrix} 2 & 0 & 0 \\ 1 & 1 & 0 \\ -1 & -2 & 3 \end{pmatrix}.$$

Solving $\mathbf{L}\boldsymbol{y} = \boldsymbol{b}$ gives $\boldsymbol{y} = \begin{pmatrix} 1/2 \\ -1/2 \\ 1/6 \end{pmatrix}$, and solving $\mathbf{L}^\top\boldsymbol{x} = \boldsymbol{y}$ gives

$$\boldsymbol{x} = \begin{pmatrix} 17/36 \\ -7/18 \\ 1/18 \end{pmatrix}.$$

Overdetermined Systems and Linear Regression

Given data $(x_1, y_1), (x_2, y_2), \ldots, (x_n, y_n)$, suppose that we want to fit a line $y = a_1 x + a_0$. The equations $a_1 x_i + a_0 = y_i$ can be written as

$$\begin{pmatrix} x_1 & 1 \\ \vdots & \vdots \\ x_n & 1 \end{pmatrix} \begin{pmatrix} a_1 \\ a_0 \end{pmatrix} = \begin{pmatrix} y_1 \\ \vdots \\ y_n \end{pmatrix},$$

or simply as $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$, where $\mathbf{A} \in \mathbb{R}^{n \times 2}$ and $\boldsymbol{b} \in \mathbb{R}^n$. Usually this system is overdetermined, so we seek $\boldsymbol{x}$ such that $\mathbf{A}\boldsymbol{x}$ is as close to $\boldsymbol{b}$ as possible. Suppose that $\boldsymbol{x}_1$ and $\boldsymbol{x}_2$ are two solution "candidates." To compare residuals such as $\mathbf{A}\boldsymbol{x}_1 - \boldsymbol{b}$ and $\mathbf{A}\boldsymbol{x}_2 - \boldsymbol{b}$, we need a norm. Define the

2-norm, the **1-norm**, and the **∞-norm** as

$$\|\boldsymbol{y}\|_2 := \sqrt{y_1^2 + \cdots + y_n^2}, \qquad \|\boldsymbol{y}\|_1 := |y_1| + \cdots + |y_n|, \qquad \text{and} \qquad \|\boldsymbol{y}\|_\infty := \max\{|y_1|, \ldots, |y_n|\},$$

respectively. We minimize the squared 2-norm of the residual vector (or rather half of it, purely for mathematical convenience). Assuming $\mathbf{A}$ has full column rank, we obtain

$$\min_{\boldsymbol{x}} \frac{1}{2}\|\mathbf{A}\boldsymbol{x} - \boldsymbol{b}\|_2^2 \iff \min_{\boldsymbol{x}} \frac{1}{2}(\mathbf{A}\boldsymbol{x} - \boldsymbol{b})^\top(\mathbf{A}\boldsymbol{x} - \boldsymbol{b})$$

and hence, after dropping the constant term, the equivalent quadratic objective is

$$q(\boldsymbol{x}) = \frac{1}{2}\boldsymbol{x}^\top\mathbf{A}^\top\mathbf{A}\boldsymbol{x} - (\mathbf{A}^\top\boldsymbol{b})^\top\boldsymbol{x}.$$

The first-order condition is then

$$\nabla q(\boldsymbol{x}) = \mathbf{A}^\top\mathbf{A}\boldsymbol{x} - \mathbf{A}^\top\boldsymbol{b} = \mathbf{0}_n.$$

Lecture 5 — Thursday, September 18, 2014

Recall that for a symmetric positive definite system $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$, the Cholesky method consists of factoring $\mathbf{A} = \mathbf{L}\mathbf{L}^\top$, solving $\mathbf{L}\boldsymbol{y} = \boldsymbol{b}$, and solving $\mathbf{L}^\top\boldsymbol{x} = \boldsymbol{y}$.

Now let $\mathbf{A} \in \mathbb{R}^{m\times n}$ with $m > n$. The least-squares problem is

$$\mathbf{A}\boldsymbol{x} \approx \boldsymbol{b} \iff \min_{\boldsymbol{x}} \frac{1}{2}\|\mathbf{A}\boldsymbol{x} - \boldsymbol{b}\|_2^2.$$

With $q(\boldsymbol{x}) = \frac{1}{2}\boldsymbol{x}^\top\mathbf{A}^\top\mathbf{A}\boldsymbol{x} - (\mathbf{A}^\top\boldsymbol{b})^\top\boldsymbol{x}$, the optimality condition is $\nabla q(\boldsymbol{x}) = \mathbf{0}_n$.

Thus

$$\mathbf{A}^\top\mathbf{A}\boldsymbol{x} = \mathbf{A}^\top\boldsymbol{b}.$$

If $\operatorname{rank}(\mathbf{A}) = n$, then $\mathbf{A}^\top\mathbf{A}$ is symmetric positive definite and the **least-squares solution** is

$$\boldsymbol{x}_{LS} = (\mathbf{A}^\top\mathbf{A})^{-1}\mathbf{A}^\top\boldsymbol{b}.$$

The **method of normal equations** is therefore:

1. Compute $\mathbf{M} = \mathbf{A}^\top \mathbf{A}$ and $\tilde{\boldsymbol{b}} = \mathbf{A}^\top \boldsymbol{b}$.
2. Compute the Cholesky factorization $\mathbf{M} = \mathbf{L}\mathbf{L}^\top$.
3. Solve $\mathbf{L}\boldsymbol{y} = \tilde{\boldsymbol{b}}$.
4. Solve $\mathbf{L}^\top \boldsymbol{x} = \boldsymbol{y}$.

QR Factorizations

Let $\mathbf{A} \in \mathbb{R}^{m \times n}$ with $m \geq n$. A **QR factorization** has the form

$$\mathbf{A} = \mathbf{Q} \begin{pmatrix} \mathbf{R} \\ \mathbf{0}_{(m-n)\times n} \end{pmatrix},$$

where $\mathbf{Q}$ is an $m \times m$ orthogonal matrix, $\mathbf{R}$ is an $n \times n$ upper triangular matrix, and $\mathbf{0}_{(m-n)\times n}$ is an $(m-n) \times n$ zero matrix. If $\operatorname{rank}(\mathbf{A}) = n$, then the diagonal entries of $\mathbf{R}$ are nonzero.

If we split $\mathbf{Q}$ into $[\mathbf{Y}, \mathbf{Z}]$ where $\mathbf{Y}$ consists of the first n columns of $\mathbf{Q}$, we have

$$\mathbf{A} = [\mathbf{Y}, \mathbf{Z}] \begin{pmatrix} \mathbf{R} \\ \mathbf{0}_{(m-n)\times n} \end{pmatrix} = \mathbf{Y}\mathbf{R}.$$

This is the **compact QR factorization**, which is much cheaper to compute than the full form. Here $\mathbf{Y}^\top \mathbf{Y} = \mathbf{I}_n$, $\mathbf{Z}^\top \mathbf{Z} = \mathbf{I}_{m-n}$, and $\mathbf{Y}^\top \mathbf{Z} = \mathbf{0}_{n\times(m-n)}$. Note that $\mathbf{Y}$ is an orthonormal matrix.

In MATLAB, `[Q,R] = qr(A)` computes the full form, while `[Y,R] = qr(A,0)` computes the compact form.

Since $\mathbf{A} = \mathbf{Y}\mathbf{R}$, the normal equations become

$$\mathbf{R}^\top \mathbf{Y}^\top \mathbf{Y}\mathbf{R}\boldsymbol{x} = \mathbf{R}^\top \mathbf{Y}^\top \boldsymbol{b}.$$

Using $\mathbf{Y}^\top \mathbf{Y} = \mathbf{I}$ and the nonsingularity of $\mathbf{R}$, this reduces to

$$\mathbf{R}\boldsymbol{x} = \mathbf{Y}^\top \boldsymbol{b}.$$

Thus the QR method for solving $\mathbf{A}\boldsymbol{x} \approx \boldsymbol{b}$ is:

1. Compute $\mathbf{A} = \mathbf{YR}$.
2. Compute $\tilde{\boldsymbol{b}} = \mathbf{Y}^\top \boldsymbol{b}$.
3. Solve $\mathbf{R}\boldsymbol{x} = \tilde{\boldsymbol{b}}$.

In MATLAB, we may write `[Y,R] = qr(A,0)`, `bbar = Y'*b`, and `x = R\bbar`. The command `x = A\b` calls an appropriate solver automatically.

The QR method is generally more numerically stable than the method of normal equations because forming $\mathbf{A}^\top\mathbf{A}$ squares the condition number. How many reliable digits this saves depends on the problem; in a poorly conditioned example, normal equations might give about 7 reliable digits while QR gives 12 or 13.

Lecture 6 — Friday, September 19, 2014

For overdetermined systems, the normal equations solve $\mathbf{A}^\top\mathbf{A}\boldsymbol{x} = \mathbf{A}^\top\boldsymbol{b}$, while the QR approach writes $\mathbf{A} = \mathbf{YR}$ and solves $\mathbf{R}\boldsymbol{x} = \mathbf{Y}^\top\boldsymbol{b}$.

Underdetermined Systems and Basic Solutions

Now consider an underdetermined system $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$, where $\mathbf{A} \in \mathbb{R}^{m\times n}$, $\boldsymbol{b} \in \mathbb{R}^m$, and $m < n$. Such systems often arise in portfolio optimization. For example, suppose that stocks $S_1, \ldots, S_n$ have returns $\mu_1, \ldots, \mu_n$, and suppose that we want a portfolio with return 5%. We may then minimize an objective $F(\boldsymbol{x})$ subject to

$$\sum_{i=1}^{n} x_i\mu_i = 5\%,$$

and

$$\boldsymbol{x}^\top\mathbf{1} = 1.$$

There may be infinitely many feasible solutions. We can describe all of them once we know one particular solution and the null space of $\mathbf{A}$. If $\boldsymbol{x}_0$ is a solution, then $\mathbf{A}\boldsymbol{x}_0 = \boldsymbol{b}$. For any $\boldsymbol{y} \in \text{Null}(\mathbf{A})$,

$$\mathbf{A}(\boldsymbol{x}_0 + \boldsymbol{y}) = \mathbf{A}\boldsymbol{x}_0 + \mathbf{A}\boldsymbol{y} = \mathbf{A}\boldsymbol{x}_0 + \mathbf{0}_m = \boldsymbol{b}.$$

Thus every vector of the form $\boldsymbol{x}_0 + \boldsymbol{y}$, with $\boldsymbol{y} \in \text{Null}(\mathbf{A})$, is also a solution. The problem is therefore to find a particular solution $\boldsymbol{x}_0$ and a basis for the null space.

One way to find such a particular solution is to use a QR factorization of $\mathbf{A}^\top$:

$$\mathbf{A}^\top = \mathbf{Q}\begin{pmatrix}\mathbf{R}\\ \mathbf{0}_{(n-m)\times m}\end{pmatrix} = \begin{pmatrix}\mathbf{Y} & \mathbf{Z}\end{pmatrix}\begin{pmatrix}\mathbf{R}\\ \mathbf{0}_{(n-m)\times m}\end{pmatrix} = \mathbf{Y}\mathbf{R} \implies \mathbf{A} = \mathbf{R}^\top\mathbf{Y}^\top.$$

Consider the following construction:

1. Solve $\mathbf{R}^\top \boldsymbol{v} = \boldsymbol{b}$.
2. Set $\boldsymbol{x}_0 = \mathbf{Y}\boldsymbol{v}$.

Then

$$\mathbf{A}\boldsymbol{x}_0 = \mathbf{A}\mathbf{Y}(\mathbf{R}^\top)^{-1}\boldsymbol{b} = \mathbf{R}^\top\mathbf{Y}^\top\mathbf{Y}(\mathbf{R}^\top)^{-1}\boldsymbol{b} = \boldsymbol{b}.$$

It follows that $\boldsymbol{x}_0$ is the shortest solution of $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$, in the sense that $\|\boldsymbol{x}\|_2$ is minimized at $\boldsymbol{x} = \boldsymbol{x}_0$ among all solutions.

For the null space, we have

$$\mathbf{A}\mathbf{Z} = \mathbf{R}^\top\mathbf{Y}^\top\mathbf{Z} = \mathbf{R}^\top\mathbf{0}_{m\times(n-m)} = \mathbf{0}_{m\times(n-m)},$$

so the columns of $\mathbf{Z}$ form a basis for Null($\mathbf{A}$). Since rank($\mathbf{Z}$) $= n - m$, every vector in Null($\mathbf{A}$) can be written as $\mathbf{Z}\boldsymbol{w}$ for some $\boldsymbol{w} \in \mathbb{R}^{n-m}$.

In summary:

1. Compute $\mathbf{A}^\top = \mathbf{Q}\begin{pmatrix}\mathbf{R}\\ \mathbf{0}_{(n-m)\times m}\end{pmatrix} = \begin{pmatrix}\mathbf{Y} & \mathbf{Z}\end{pmatrix}\begin{pmatrix}\mathbf{R}\\ \mathbf{0}_{(n-m)\times m}\end{pmatrix}$.
2. Solve $\mathbf{R}^\top \boldsymbol{v} = \boldsymbol{b}$.
3. Set $\boldsymbol{x}_0 = \mathbf{Y}\boldsymbol{v}$.
4. Write the general solution as $\boldsymbol{x} = \boldsymbol{x}_0 + \mathbf{Z}\boldsymbol{w}$.

Another way to obtain a particular solution is to partition $\mathbf{A} = [\mathbf{A}_1, \mathbf{A}_2]$ and assume first that $\mathbf{A}_1$ is nonsingular. Then a particular solution of $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$ is obtained as follows:

1. Solve $\mathbf{A}_1\boldsymbol{x}_1 = \boldsymbol{b}$, for instance by LU factorization.
2. Set $\boldsymbol{x}_0 = \begin{pmatrix}\boldsymbol{x}_1\\ \mathbf{0}_{n-m}\end{pmatrix}$.

Construct a basis of Null($\mathbf{A}$) by setting

$$\mathbf{Z} = \begin{pmatrix} -\mathbf{A}_1^{-1}\mathbf{A}_2 \\ \mathbf{I}_{n-m} \end{pmatrix}.$$

Indeed,

$$\mathbf{AZ} = (\mathbf{A}_1, \mathbf{A}_2)\begin{pmatrix} -\mathbf{A}_1^{-1}\mathbf{A}_2 \\ \mathbf{I}_{n-m} \end{pmatrix} = -\mathbf{A}_2 + \mathbf{A}_2 = \mathbf{0}_{m\times(n-m)}.$$

Also, rank($\mathbf{Z}$) $= n - m$.

Lecture 7 — Tuesday, September 23, 2014

We are trying to solve the underdetermined system $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$, where $\mathbf{A} \in \mathbb{R}^{m\times n}$, $\boldsymbol{b} \in \mathbb{R}^m$, and $m < n$. Our current approach writes $\mathbf{A} = (\mathbf{A}_1, \mathbf{A}_2)$ and does the following:

1. Solve $\mathbf{A}_1\boldsymbol{x}_1 = \boldsymbol{b}$.
2. Set $\boldsymbol{x}_0 = \begin{pmatrix} \boldsymbol{x}_1 \\ \mathbf{0}_{n-m} \end{pmatrix}$, $\mathbf{Z} = \begin{pmatrix} -\mathbf{A}_1^{-1}\mathbf{A}_2 \\ \mathbf{I}_{n-m} \end{pmatrix}$.

If $\mathbf{A}_1$ is singular, we can introduce a permutation matrix $\mathbf{P}$ such that

$$\mathbf{AP} = (\mathbf{A}_1, \mathbf{A}_2)\mathbf{P} = (\tilde{\mathbf{A}}_1, \tilde{\mathbf{A}}_2),$$

where $\tilde{\mathbf{A}}_1$ is nonsingular. One way to find $\mathbf{P}$ is to factor $\mathbf{A}^\top$:

$$\mathbf{PA}^\top = \mathbf{LU} = \begin{pmatrix} \mathbf{L}_1 \\ \mathbf{L}_2 \end{pmatrix}\mathbf{U} \implies \mathbf{AP}^\top = \mathbf{U}^\top(\mathbf{L}_1^\top, \mathbf{L}_2^\top) = (\mathbf{U}^\top\mathbf{L}_1^\top, \mathbf{U}^\top\mathbf{L}_2^\top).$$

1. Solve $\mathbf{AP}^\top \begin{pmatrix} \boldsymbol{x}_1 \\ \mathbf{0}_{n-m} \end{pmatrix} = \boldsymbol{b}$. Since $\mathbf{U}^\top\mathbf{L}_1^\top\boldsymbol{x}_1 = \boldsymbol{b}$, solve $\mathbf{U}^\top\boldsymbol{y} = \boldsymbol{b}$ and $\mathbf{L}_1^\top\boldsymbol{x}_1 = \boldsymbol{y}$.
2. $\boldsymbol{x}_0 = \mathbf{P}^\top \begin{pmatrix} \boldsymbol{x}_1 \\ \mathbf{0}_{n-m} \end{pmatrix}$.
3. Set $\mathbf{Z} = \mathbf{P}^\top \begin{pmatrix} -\mathbf{L}_1^{-\top}\mathbf{L}_2^\top \\ \mathbf{I}_{n-m} \end{pmatrix}$, so rank($\mathbf{Z}$) $= n - m$.

This choice of $\mathbf{Z}$ satisfies

$$\mathbf{AZ} = \mathbf{AP}^\top \begin{pmatrix} -\mathbf{L}_1^{-\top}\mathbf{L}_2^\top \\ \mathbf{I}_{n-m} \end{pmatrix} = \mathbf{U}^\top(\mathbf{L}_1^\top, \mathbf{L}_2^\top) \begin{pmatrix} -\mathbf{L}_1^{-\top}\mathbf{L}_2^\top \\ \mathbf{I}_{n-m} \end{pmatrix} = \mathbf{U}^\top(-\mathbf{L}_2^\top + \mathbf{L}_2^\top) = \mathbf{0}_{m\times(n-m)}.$$

In summary, the **basic solution method** for $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$ produces a solution as follows:

1. LU factor $\mathbf{A}^\top$ as $\mathbf{PA}^\top = \mathbf{LU} = \begin{pmatrix} \mathbf{L}_1 \\ \mathbf{L}_2 \end{pmatrix} \mathbf{U}$.
2. Solve $\mathbf{L}_1^\top \boldsymbol{y} = \boldsymbol{b}$ and $\mathbf{U}^\top \boldsymbol{x}_1 = \boldsymbol{y}$.
3. $\boldsymbol{x}_0 = \mathbf{P}^\top \begin{pmatrix} \boldsymbol{x}_1 \\ \mathbf{0}_{n-m} \end{pmatrix}$.
4. Construct $\mathbf{Z} = \mathbf{P}^\top \begin{pmatrix} -\mathbf{L}_1^{-\top}\mathbf{L}_2^\top \\ \mathbf{I}_{n-m} \end{pmatrix}$.
5. Construct any feasible point $\boldsymbol{x}_0 + \mathbf{Z}\boldsymbol{w}$, for any $\boldsymbol{w} \in \mathbb{R}^{n-m}$.

Lecture 8 — Thursday, September 25, 2014

Low-Rank Updates and Block Solvers

For a constrained problem

$$\min f(\boldsymbol{x}) \quad \text{subject to } \mathbf{A}\boldsymbol{x} = \boldsymbol{b},$$

we can substitute the general feasible form $\boldsymbol{x} = \boldsymbol{x}_0 + \mathbf{Z}\boldsymbol{w}$ and solve the unconstrained problem

$$\min_{\boldsymbol{w}} f(\boldsymbol{x}_0 + \mathbf{Z}\boldsymbol{w}).$$

Low-rank updates arise when a factorization is already known and the matrix changes by a low-rank matrix. For example, given a QR or Cholesky factorization of $\mathbf{A}$, we may seek a factorization of $\mathbf{A} + \mathbf{B}$ efficiently when $\operatorname{rank}(\mathbf{B})$ is small. The official notes emphasize that such updates can reduce the dominant work from about $O(n^3)$ to about $O(n^2)$ in favourable cases.

Suppose that $\mathbf{H} = \mathbf{W} + \mathbf{V}\mathbf{V}^\top$, where $\mathbf{V} \in \mathbb{R}^{n\times t}$ with $t \ll n$, and suppose that systems involving

$\mathbf{W}$ can be solved quickly. The system $\mathbf{H}\boldsymbol{x} = \boldsymbol{r}$ is equivalent to

$$\begin{pmatrix} \mathbf{W} & \mathbf{V} \\ \mathbf{V}^\top & -\mathbf{I}_t \end{pmatrix} \begin{pmatrix} \boldsymbol{x} \\ \boldsymbol{z} \end{pmatrix} = \begin{pmatrix} \boldsymbol{r} \\ \mathbf{0}_t \end{pmatrix}$$

because this block system gives

$$\begin{aligned} \mathbf{W}\boldsymbol{x} + \mathbf{V}\boldsymbol{z} &= \boldsymbol{r}, \\ \mathbf{V}^\top \boldsymbol{x} - \boldsymbol{z} &= \mathbf{0}_t. \end{aligned}$$

Multiplying the first equation by $-V^\top W^{-1}$ gives

$$-\mathbf{V}^\top \mathbf{W}^{-1}(\mathbf{W}\boldsymbol{x} + \mathbf{V}\boldsymbol{z}) = -\mathbf{V}^\top \mathbf{W}^{-1}\boldsymbol{r}$$

and hence

$$-\mathbf{V}^\top \boldsymbol{x} - \mathbf{V}^\top \mathbf{W}^{-1}\mathbf{V}\boldsymbol{z} = -\mathbf{V}^\top \mathbf{W}^{-1}\boldsymbol{r}$$

Lecture 9 — Friday, September 26, 2014

For an underdetermined system $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$, with $\mathbf{A} \in \mathbb{R}^{m \times n}$, $\boldsymbol{b} \in \mathbb{R}^m$, $m < n$, and $\operatorname{rank}(\mathbf{A}) = m$, the basic solution construction can be summarized as follows:

1. LU factorize $\mathbf{A}^\top$: $\mathbf{P}\mathbf{A}^\top = \mathbf{L}\mathbf{U} = \begin{pmatrix} \mathbf{L}_1 \\ \mathbf{L}_2 \end{pmatrix} \mathbf{U}$.
2. Solve $\mathbf{U}^\top \boldsymbol{y} = \boldsymbol{b}$ and $\mathbf{L}_1^\top \boldsymbol{x}_1 = \boldsymbol{y}$.
3. $\boldsymbol{x}_0 = \mathbf{P}^\top \begin{pmatrix} \boldsymbol{x}_1 \\ \mathbf{0}_{n-m} \end{pmatrix}$ and $\mathbf{Z} = \mathbf{P}^\top \begin{pmatrix} -\mathbf{L}_1^{-\top}\mathbf{L}_2^\top \\ \mathbf{I}_{n-m} \end{pmatrix}$.

For the block solver, suppose $\mathbf{H} = \mathbf{W} + \mathbf{V}\mathbf{V}^\top$, where $\mathbf{V} \in \mathbb{R}^{n \times t}$ with $t \ll n$, and $\mathbf{W} \in \mathbb{R}^{n \times n}$ is symmetric positive definite. Then

$$\mathbf{H}\boldsymbol{x} = \boldsymbol{r} \iff \begin{pmatrix} \mathbf{W} & \mathbf{V} \\ \mathbf{V}^\top & -\mathbf{I}_t \end{pmatrix} \begin{pmatrix} \boldsymbol{x} \\ \boldsymbol{z} \end{pmatrix} = \begin{pmatrix} \boldsymbol{r} \\ \mathbf{0}_t \end{pmatrix}.$$

Eliminating $\boldsymbol{x}$ gives

$$\begin{pmatrix} \mathbf{W} & \mathbf{V} \\ \mathbf{0}_{t\times n} & -\mathbf{I}_t - \mathbf{V}^\top \mathbf{W}^{-1}\mathbf{V} \end{pmatrix} \begin{pmatrix} \boldsymbol{x} \\ \boldsymbol{z} \end{pmatrix} = \begin{pmatrix} \boldsymbol{r} \\ -\mathbf{V}^\top \mathbf{W}^{-1}\boldsymbol{r} \end{pmatrix}.$$

1. Solve $(\mathbf{I}_t + \mathbf{V}^\top \mathbf{W}^{-1}\mathbf{V})\boldsymbol{z} = \mathbf{V}^\top \mathbf{W}^{-1}\boldsymbol{r}$, which is a $t \times t$ system.
2. Solve $\mathbf{W}\boldsymbol{x} = \boldsymbol{r} - \mathbf{V}\boldsymbol{z}$.

When solves with $\mathbf{W}$ are fast, the reduced solve costs $O(t^3)$, while forming and applying the dense low-rank terms typically costs $O(nt^2)$ in addition to the required solves with $\mathbf{W}$. For fixed t, the work can therefore scale linearly with n rather than cubically.

A special case is $\mathbf{V} = \boldsymbol{v} \in \mathbb{R}^n$ and $\mathbf{W} = \mathbf{T}$, where $\mathbf{T}$ is tridiagonal. Then $\mathbf{H}\boldsymbol{x} = (\boldsymbol{v}\boldsymbol{v}^\top + \mathbf{T})\boldsymbol{x} = \boldsymbol{r}$, and the computation is as follows:

1. Solve $\mathbf{T}\boldsymbol{w} = \boldsymbol{v}$, and compute $s = \boldsymbol{v}^\top \boldsymbol{w}$ and $p = \boldsymbol{r}^\top \boldsymbol{w}$.
2. Compute $z = p/(1+s)$.
3. Solve $\mathbf{T}\boldsymbol{x} = \boldsymbol{r} - \boldsymbol{v}z$.

Example 9.1 Let

$$\mathbf{T} = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}, \qquad \boldsymbol{v} = \begin{pmatrix} 1 \\ 1 \end{pmatrix}, \qquad \text{and} \qquad \boldsymbol{r} = \begin{pmatrix} 1 \\ 0 \end{pmatrix}.$$

Then $\mathbf{H} = \mathbf{T} + \boldsymbol{v}\boldsymbol{v}^\top$ is

$$\mathbf{H} = \mathbf{T} + \boldsymbol{v}\boldsymbol{v}^\top = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix} + \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix} = \begin{pmatrix} 3 & 2 \\ 2 & 3 \end{pmatrix}.$$

Solving $\mathbf{T}\boldsymbol{w} = \boldsymbol{v}$ gives $\boldsymbol{w} = \begin{pmatrix} 1/3 \\ 1/3 \end{pmatrix}$, so $s = \boldsymbol{v}^\top \boldsymbol{w} = 2/3$ and $p = \boldsymbol{r}^\top \boldsymbol{w} = 1/3$. Hence $\boldsymbol{z} = p/(1+s) = 1/5$. Finally, solving

$$\mathbf{T}\boldsymbol{x} = \boldsymbol{r} - \boldsymbol{v}\boldsymbol{z} = \begin{pmatrix} 4/5 \\ -1/5 \end{pmatrix}$$

gives

$$\boldsymbol{x} = \begin{pmatrix} 3/5 \\ -2/5 \end{pmatrix}.$$

Example 9.2 Let

$$\mathbf{T} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 2 & 0 & 0 \\ 0 & 0 & 3 & 0 \\ 0 & 0 & 0 & 4 \end{pmatrix}, \qquad \boldsymbol{v} = \begin{pmatrix} 1 \\ 1 \\ 1 \\ 1 \end{pmatrix}, \qquad \text{and} \qquad \boldsymbol{r} = \begin{pmatrix} 1 \\ 0 \\ 0 \\ 0 \end{pmatrix}.$$

We want to solve

$$(\mathbf{T} + \boldsymbol{v}\boldsymbol{v}^\top)\boldsymbol{x} = \boldsymbol{r}$$

using the block solver steps above.

First solve $\mathbf{T}\boldsymbol{w} = \boldsymbol{v}$:

$$\boldsymbol{w} = \begin{pmatrix} 1 \\ 1/2 \\ 1/3 \\ 1/4 \end{pmatrix}.$$

Then

$$s = \boldsymbol{v}^\top \boldsymbol{w} = 1 + \frac{1}{2} + \frac{1}{3} + \frac{1}{4} = \frac{25}{12} \qquad \text{and} \qquad p = \boldsymbol{r}^\top \boldsymbol{w} = 1.$$

Hence

$$z = \frac{p}{1+s} = \frac{1}{1 + 25/12} = \frac{12}{37}.$$

Now solve

$$\mathbf{T}\boldsymbol{x} = \boldsymbol{r} - \boldsymbol{v}z = \begin{pmatrix} 1 \\ 0 \\ 0 \\ 0 \end{pmatrix} - \frac{12}{37}\begin{pmatrix} 1 \\ 1 \\ 1 \\ 1 \end{pmatrix} = \begin{pmatrix} 25/37 \\ -12/37 \\ -12/37 \\ -12/37 \end{pmatrix}.$$

Since $\mathbf{T}$ is diagonal,

$$\boldsymbol{x} = \begin{pmatrix} 25/37 \\ -6/37 \\ -4/37 \\ -3/37 \end{pmatrix}$$

is our solution.

Now consider the constrained block system

$$\begin{pmatrix} \mathbf{H} & \mathbf{A}^\top \\ \mathbf{A} & \mathbf{0}_{m\times m} \end{pmatrix} \begin{pmatrix} \boldsymbol{x} \\ \boldsymbol{y} \end{pmatrix} = \begin{pmatrix} \boldsymbol{r} \\ \boldsymbol{b} \end{pmatrix},$$

where $\mathbf{H} = \mathbf{W} + \mathbf{V}\mathbf{V}^\top$ and $\mathbf{A} \in \mathbb{R}^{m\times n}$. Define $\boldsymbol{z} = \mathbf{V}^\top \boldsymbol{x}$.

$$\begin{pmatrix} \mathbf{W} & \mathbf{V} & \mathbf{A}^\top \\ \mathbf{V}^\top & -\mathbf{I}_t & \mathbf{0}_{t\times m} \\ \mathbf{A} & \mathbf{0}_{m\times t} & \mathbf{0}_{m\times m} \end{pmatrix} \begin{pmatrix} \boldsymbol{x} \\ \boldsymbol{z} \\ \boldsymbol{y} \end{pmatrix} = \begin{pmatrix} \boldsymbol{r} \\ \mathbf{0}_t \\ \boldsymbol{b} \end{pmatrix}$$

is equivalent, after eliminating through $\mathbf{W}$, to

$$\begin{pmatrix} \mathbf{W} & \mathbf{V} & \mathbf{A}^\top \\ \mathbf{0}_{t\times n} & -\mathbf{I}_t - \mathbf{V}^\top\mathbf{W}^{-1}\mathbf{V} & -\mathbf{V}^\top\mathbf{W}^{-1}\mathbf{A}^\top \\ \mathbf{0}_{m\times n} & -\mathbf{A}\mathbf{W}^{-1}\mathbf{V} & -\mathbf{A}\mathbf{W}^{-1}\mathbf{A}^\top \end{pmatrix} \begin{pmatrix} \boldsymbol{x} \\ \boldsymbol{z} \\ \boldsymbol{y} \end{pmatrix} = \begin{pmatrix} \boldsymbol{r} \\ -\mathbf{V}^\top\mathbf{W}^{-1}\boldsymbol{r} \\ \boldsymbol{b} - \mathbf{A}\mathbf{W}^{-1}\boldsymbol{r} \end{pmatrix}.$$

So for this setup, we perform the following steps:

1. Solve $\bar{\mathbf{W}} \begin{pmatrix} \boldsymbol{z} \\ \boldsymbol{y} \end{pmatrix} = \boldsymbol{d}$.
2. Solve $\mathbf{W}\boldsymbol{x} = \boldsymbol{r} - \mathbf{V}\boldsymbol{z} - \mathbf{A}^\top\boldsymbol{y}$.

The total cost is $O((m+t)^3) + O(tn) + O(mn) + O(n)$, compared with $O((m+n)^3)$ for a direct dense solve. For example, if

$$m = 6, \qquad t = 4, \qquad \text{and} \qquad n = 100.$$

then $m + t = 10$, while a direct method would involve size $m + n = 106$. Comparing only the cubic terms, the block method requires roughly three orders of magnitude less work than a dense Gaussian-elimination solve; the actual speedup also depends on the matrix structure and implementation.

Lecture 10 — Tuesday, September 30, 2014

Suppose $\mathbf{W}\boldsymbol{s} = \boldsymbol{p}$ has a fast linear solver. To solve $\mathbf{H}\boldsymbol{x} = \boldsymbol{r}$, where $\mathbf{H} = \mathbf{W} + \boldsymbol{v}\boldsymbol{v}^\top$, use the block form

$$\begin{pmatrix} \mathbf{W} & \boldsymbol{v} \\ \boldsymbol{v}^\top & -\mathbf{I}_1 \end{pmatrix} \begin{pmatrix} \boldsymbol{x} \\ z \end{pmatrix} = \begin{pmatrix} \boldsymbol{r} \\ \mathbf{0}_1 \end{pmatrix}.$$

After elimination this becomes

$$\begin{pmatrix} \mathbf{W} & \boldsymbol{v} \\ \mathbf{0}_{1\times n} & \mathbf{I}_1 + \boldsymbol{v}^\top \mathbf{W}^{-1}\boldsymbol{v} \end{pmatrix} \begin{pmatrix} \boldsymbol{x} \\ z \end{pmatrix} = \begin{pmatrix} \boldsymbol{r} \\ \boldsymbol{v}^\top \mathbf{W}^{-1}\boldsymbol{r} \end{pmatrix}.$$

The computational steps are:

1. Solve $\mathbf{W}\bar{\boldsymbol{v}} = \boldsymbol{v}$.
2. Solve $\mathbf{W}\bar{\boldsymbol{r}} = \boldsymbol{r}$.
3. Compute $z = (1 + \boldsymbol{v}^\top \bar{\boldsymbol{v}})^{-1} \boldsymbol{v}^\top \bar{\boldsymbol{r}}$.
4. Set $\boldsymbol{x} = \bar{\boldsymbol{r}} - \bar{\boldsymbol{v}} z$.

Spectral Decompositions

If $\mathbf{A} \in \mathbb{R}^{n\times n}$ and $\mathbf{A} = \mathbf{A}^\top$, then $\mathbf{A}$ is diagonalizable. This means that we can write

$$\mathbf{A} = \mathbf{U}\boldsymbol{\Sigma}\mathbf{U}^\top,$$

where $\mathbf{U}$ is orthogonal and $\boldsymbol{\Sigma} = \operatorname{diag}(\lambda_1, \ldots, \lambda_n)$ with $\lambda_i \in \mathbb{R}$. Also, $\mathbf{A}\boldsymbol{u}_i = \lambda_i \boldsymbol{u}_i$, where $\boldsymbol{u}_i$ is the ith column of $\mathbf{U}$ and hence the ith eigenvector.

In MATLAB, `eig(A)` returns the eigenvalues, while `[U, D] = eig(A)` returns the decomposition.

The full decomposition is expensive, so it is used only when necessary. In many applications, it is enough to compute only the extreme eigenvalues, such as the largest and smallest eigenvalues.

If $\mathbf{A}$ is symmetric positive definite, then $\lambda_i > 0$ for all i. If $\mathbf{A}$ is positive semidefinite, then $\lambda_i \geq 0$ for all i. For any $\mathbf{A} \in \mathbb{R}^{m\times n}$ (symmetric or otherwise), there is a **singular value decomposition**

$$\mathbf{A} = \mathbf{U}\boldsymbol{\Sigma}\mathbf{V}^\top,$$

where $\mathbf{U} \in \mathbb{R}^{m\times m}$ and $\mathbf{V} \in \mathbb{R}^{n\times n}$ are orthogonal, and $\mathbf{\Sigma} \in \mathbb{R}^{m\times n}$ is diagonal except for possible zero padding. Its nonzero diagonal entries are the singular values

$$\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_r > 0.$$

The singular value decomposition exists for every matrix, but naturally it is expensive to compute, especially compared with LU, QR, or Cholesky factorizations. Its advantage is that it gives reliable information about rank, range, and null spaces. In particular, rank($\mathbf{A}$) is the number of nonzero singular values, the first r columns of $\mathbf{U}$ span Range($\mathbf{A}$), and the last $n-r$ columns of $\mathbf{V}$ span Null($\mathbf{A}$).

For any $k < r$, let $\mathbf{U}_k$, $\mathbf{\Sigma}_k$, and $\mathbf{V}_k$ denote the leading rank-k pieces. Then

$$\mathbf{A}_k = \mathbf{U}_k \mathbf{\Sigma}_k \mathbf{V}_k^\top$$

is the best rank-k approximation to $\mathbf{A}$ in Frobenius norm. In MATLAB, `d = svd(A)` returns the singular values and `[U,D,V] = svd(A)` returns the full decomposition.

The singular value decomposition is enormously useful, but also expensive. Use it only if you are loaded (computationally speaking).

2. Computing Derivatives

First Derivatives and Taylor Approximation

Let

$$F(\boldsymbol{x}) = \begin{pmatrix} f_1(\boldsymbol{x}) \\ \vdots \\ f_m(\boldsymbol{x}) \end{pmatrix} \qquad \text{and} \qquad f_i : \mathbb{R}^n \to \mathbb{R}.$$

Thus $F : \mathbb{R}^n \to \mathbb{R}^m$. The matrix of first derivatives is the **Jacobian**

$$\mathbf{J}(\boldsymbol{x}) = \begin{pmatrix} \frac{\partial f_1}{\partial x_1} & \cdots & \frac{\partial f_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial x_1} & \cdots & \frac{\partial f_m}{\partial x_n} \end{pmatrix} \in \mathbb{R}^{m\times n}.$$

When $m = 1$, the Jacobian is the transpose of the gradient:

$$\mathbf{J}(\boldsymbol{x}) = \begin{pmatrix} \frac{\partial f}{\partial x_1} & \dots & \frac{\partial f}{\partial x_n} \end{pmatrix} = \nabla f(\boldsymbol{x})^\top.$$

Lecture 11 — Thursday, October 02, 2014

First derivatives represent rates of change. For a scalar objective function $f : \mathbb{R}^n \to \mathbb{R}$, the gradient $\nabla f(\bar{\boldsymbol{x}})$ points in the local direction of steepest ascent at $\bar{\boldsymbol{x}}$, while $-\nabla f(\bar{\boldsymbol{x}})$ points in the local direction of steepest descent.

Taylor's theorem says that for any direction $\boldsymbol{d} \in \mathbb{R}^n$ with $\|\boldsymbol{d}\|_2 = 1$ and any continuously differentiable $F : \mathbb{R}^n \to \mathbb{R}^m$,

$$F(\boldsymbol{x} + \alpha \boldsymbol{d}) = F(\boldsymbol{x}) + \alpha \mathbf{J}(\boldsymbol{x})\boldsymbol{d} + o(\alpha).$$

Here $g(\alpha) = o(\alpha)$ means that $g(\alpha)/\alpha \to 0$ as $\alpha \to 0$. For example, $\alpha^2 = o(\alpha)$, while $\sin(\alpha)$ is not $o(\alpha)$.

Proposition 11.1 Suppose that $f : \mathbb{R}^n \to \mathbb{R}$, $\|\boldsymbol{d}\|_2 = 1$, and $\boldsymbol{d}^\top \nabla f(\boldsymbol{x}) > 0$. Then $\boldsymbol{d}$ is a unit ascent direction for f at $\boldsymbol{x}$, meaning that $f(\boldsymbol{x} + \alpha \boldsymbol{d}) > f(\boldsymbol{x})$ for all sufficiently small positive α.

Proof. Taylor's theorem gives

$$f(\boldsymbol{x} + \alpha \boldsymbol{d}) = f(\boldsymbol{x}) + \alpha \nabla f(\boldsymbol{x})^\top \boldsymbol{d} + o(\alpha).$$

Thus

$$\frac{f(\boldsymbol{x} + \alpha \boldsymbol{d}) - f(\boldsymbol{x})}{\alpha} = \nabla f(\boldsymbol{x})^\top \boldsymbol{d} + \frac{o(\alpha)}{\alpha} \to \nabla f(\boldsymbol{x})^\top \boldsymbol{d} > 0$$

as $\alpha \to 0$. Hence $f(\boldsymbol{x} + \alpha \boldsymbol{d}) - f(\boldsymbol{x}) > 0$ for all sufficiently small positive α. □

Similarly, if $\boldsymbol{d}^\top \nabla f(\boldsymbol{x}) < 0$, then $\boldsymbol{d}$ is a unit descent direction for f at $\boldsymbol{x}$.

If $F : \mathbb{R}^n \to \mathbb{R}^m$ is C^1, then

$$F(\boldsymbol{x} + \alpha \boldsymbol{d}) = F(\boldsymbol{x}) + \alpha \mathbf{J}(\boldsymbol{x})\boldsymbol{d} + o(\alpha).$$

This is the **general linear approximation**.

If $f : \mathbb{R}^n \to \mathbb{R}$ is twice continuously differentiable, then

$$f(\boldsymbol{x} + \alpha \boldsymbol{d}) = f(\boldsymbol{x}) + \alpha \nabla f(\boldsymbol{x})^\top \boldsymbol{d} + \frac{1}{2}\alpha^2 \boldsymbol{d}^\top \nabla^2 f(\boldsymbol{x}) \boldsymbol{d} + o(\alpha^2),$$

which is the **general quadratic approximation**. Here

$$\nabla^2 f(\boldsymbol{x}) = \begin{pmatrix} \dfrac{\partial^2 f}{\partial x_1^2} & \cdots & \dfrac{\partial^2 f}{\partial x_1 \partial x_n} \\ \vdots & \ddots & \vdots \\ \dfrac{\partial^2 f}{\partial x_n \partial x_1} & \cdots & \dfrac{\partial^2 f}{\partial x_n^2} \end{pmatrix}$$

is the **Hessian matrix**, which is symmetric under the usual smoothness assumptions.

Nonlinear Least Squares

Let $F : \mathbb{R}^n \to \mathbb{R}^m$, and consider the **nonlinear least squares** problem

$$\min_{\boldsymbol{x}} \frac{1}{2} \|F(\boldsymbol{x})\|_2^2.$$

Equivalently, let

$$f(\boldsymbol{x}) = \frac{1}{2} F(\boldsymbol{x})^\top F(\boldsymbol{x}).$$

Then

$$\nabla f(\boldsymbol{x}) = \mathbf{J}(\boldsymbol{x})^\top F(\boldsymbol{x}),$$

and the Hessian has the structure

$$\mathbf{H}(\boldsymbol{x}) = \mathbf{J}(\boldsymbol{x})^\top \mathbf{J}(\boldsymbol{x}) + \sum_{i=1}^{m} F_i(\boldsymbol{x}) \nabla^2 F_i(\boldsymbol{x}).$$

When the residuals $F_i(\boldsymbol{x})$ are small, it is natural to use the approximation

$$\mathbf{H}(\boldsymbol{x}) \approx \mathbf{J}(\boldsymbol{x})^\top \mathbf{J}(\boldsymbol{x}).$$

This gives the **Gauss–Newton method**. A related approximation is

$$\mathbf{H}(\boldsymbol{x}) \approx \mathbf{J}(\boldsymbol{x})^\top \mathbf{J}(\boldsymbol{x}) + \lambda \mathbf{I},$$

which leads to the **Levenberg–Marquardt method** (or **damped least squares**).

Lecture 12 — Friday, October 03, 2014

Finite Difference Approximations

For $f : \mathbb{R}^n \to \mathbb{R}$, let $\boldsymbol{e}_i$ be the ith standard basis vector. Taylor's theorem gives us

$$f(\boldsymbol{x} + \alpha_i \boldsymbol{e}_i) = f(\boldsymbol{x}) + \alpha_i \nabla f(\boldsymbol{x})^\top \boldsymbol{e}_i + o(\alpha_i).$$

Rearranging gives the ith entry of the gradient:

$$\frac{f(\boldsymbol{x} + \alpha_i \boldsymbol{e}_i) - f(\boldsymbol{x})}{\alpha_i} = \nabla f(\boldsymbol{x})^\top \boldsymbol{e}_i + \frac{o(\alpha_i)}{\alpha_i} \to \nabla f(\boldsymbol{x})^\top \boldsymbol{e}_i$$

as $\alpha_i \to 0$.

The resulting **finite difference method** to approximate the gradient is:

1. For $i = 1, \ldots, n$, choose a step size α_i.
2. Set
$$g_i(\boldsymbol{x}) = \frac{f(\boldsymbol{x} + \alpha_i \boldsymbol{e}_i) - f(\boldsymbol{x})}{\alpha_i}.$$

Then $\boldsymbol{g}(\boldsymbol{x}) = (g_1(\boldsymbol{x}), \ldots, g_n(\boldsymbol{x}))^\top$ approximates the gradient of f at $\boldsymbol{x}$. The cost of this approximation is $(n + 1)$ evaluations of f.

The step size α_i balances two competing errors. If α_i is too large, truncation error dominates because the Taylor approximation is too crude. If α_i is too small, roundoff error dominates because the subtraction $f(\boldsymbol{x} + \alpha_i \boldsymbol{e}_i) - f(\boldsymbol{x})$ loses precision. This is why choosing α_i near machine precision, such as 10^{-16} in double precision, is usually too small.

Similarly, we can approximate the Jacobian matrix for $F : \mathbb{R}^n \to \mathbb{R}^m$. If $\|\boldsymbol{d}\|_2 = 1$, then

$$F(\boldsymbol{x} + \alpha \boldsymbol{d}) = F(\boldsymbol{x}) + \alpha \mathbf{J}(\boldsymbol{x}) \cdot \boldsymbol{d} + o(\alpha)$$

as $\alpha \to 0$. Thus

$$\mathbf{J}(\boldsymbol{x})\boldsymbol{d} \approx \frac{F(\boldsymbol{x} + \alpha \boldsymbol{d}) - F(\boldsymbol{x})}{\alpha}.$$

Taking $\boldsymbol{d} = \boldsymbol{e}_i$ gives the ith column of $\mathbf{J}(\boldsymbol{x})$. Computing all columns this way costs $(n + 1)$

evaluations of F.

If $\nabla f(\boldsymbol{x})$ is available, then Hessian–vector products can be approximated by differencing gradients. With $\boldsymbol{d} = \boldsymbol{e}_i$,

$$\mathbf{H}(\boldsymbol{x}) \cdot \boldsymbol{d} \approx \frac{\nabla f(\boldsymbol{x} + \alpha \boldsymbol{d}) - \nabla f(\boldsymbol{x})}{\alpha}.$$

Example 12.1 Let $F(\boldsymbol{x}) : \mathbb{R}^3 \to \mathbb{R}^3$ where

$$F(\boldsymbol{x}) = \begin{pmatrix} (3 - 2x_1)x_1 - 2x_2 + 1 \\ (3 - 2x_2)x_2 - x_1 - 2x_3 + 1 \\ (3 - 2x_3)x_3 - x_2 + 1 \end{pmatrix}.$$

An easy calculation gives

$$\mathbf{J}(\boldsymbol{x}) = \begin{pmatrix} 3 - 4x_1 & -2 & 0 \\ -1 & 3 - 4x_2 & -2 \\ 0 & -1 & 3 - 4x_3 \end{pmatrix}.$$

If $\boldsymbol{x} = (1, 1, 1)^\top$, then $F(\boldsymbol{x}) = (0, -1, 1)^\top$. Using $\alpha = 0.005$ and $\bar{\boldsymbol{x}} = \boldsymbol{x} + \alpha \boldsymbol{e}_1 = (1.005, 1, 1)^\top$ gives

$$\mathbf{J}(\boldsymbol{x}) \cdot \boldsymbol{e}_1 \approx \frac{F(\bar{\boldsymbol{x}}) - F(\boldsymbol{x})}{\alpha} = \begin{pmatrix} -1.0100 \\ -1.000 \\ 0 \end{pmatrix},$$

which is not far from the true value of

$$\mathbf{J}(\boldsymbol{x}) \cdot \boldsymbol{e}_1 = \begin{pmatrix} -1 \\ -1 \\ 0 \end{pmatrix}.$$

Lecture 13 — Tuesday, October 07, 2014

The finite-difference method approximates Jacobian–vector products by

$$\mathbf{J}(\boldsymbol{x})\boldsymbol{d} \approx \frac{F(\boldsymbol{x} + \alpha \boldsymbol{d}) - F(\boldsymbol{x})}{\alpha}.$$

If $\boldsymbol{d} = \boldsymbol{e}_i$, this recovers the ith Jacobian column. Similarly, if $\nabla f(\boldsymbol{x})$ is available, then

$$\mathbf{H}(\boldsymbol{x})\boldsymbol{d} \approx \frac{\nabla f(\boldsymbol{x} + \alpha \boldsymbol{d}) - \nabla f(\boldsymbol{x})}{\alpha}.$$

The same tradeoff in choosing the step size appears here: too small a step leads to roundoff error, while too large a step leads to truncation error.

Automatic Differentiation

Automatic differentiation (**AD**) computes derivatives by applying the chain rule to the elementary operations used to evaluate a function. It is not the same as symbolic differentiation: AD differentiates the executed computation rather than producing a closed-form derivative formula. See `www.autodiff.org` for background.

For example, let $f(x) = \sin\left(e^{x^2}\right)$ and evaluate at $x = 0.5$. In forward-mode AD, we carry each intermediate quantity together with its derivative with respect to the input. We therefore initialize

$$x.\,\mathrm{val} = 0.5 \qquad \text{and} \qquad x.\,\mathrm{deriv} = 1.$$

Now compute each operation in order.

$$\begin{aligned} y_1 &= x^2, \\ y_1.\,\mathrm{val} &= (x.\,\mathrm{val})^2 = (0.5)^2 = 0.25, \\ y_1.\,\mathrm{deriv} &= 2\,x.\,\mathrm{val}\; x.\,\mathrm{deriv} = 2(0.5)(1) = 1. \end{aligned}$$

$$\begin{aligned} y_2 &= e^{y_1}, \\ y_2.\,\mathrm{val} &= e^{y_1.\,\mathrm{val}} = e^{0.25} \approx 1.2840, \\ y_2.\,\mathrm{deriv} &= e^{y_1.\,\mathrm{val}}\, y_1.\,\mathrm{deriv} \approx 1.2840 \cdot 1 = 1.2840. \end{aligned}$$

$$\begin{aligned} y_3 &= \sin(y_2), \\ y_3.\,\mathrm{val} &= \sin\big(y_2.\,\mathrm{val}\big) = \sin(1.2840) \approx 0.9594, \\ y_3.\,\mathrm{deriv} &= \cos\big(y_2.\,\mathrm{val}\big)\, y_2.\,\mathrm{deriv} \approx \cos(1.2840) \cdot 1.2840 \approx 0.3639. \end{aligned}$$

Hence

$$f(0.5) = y_3.\,\mathrm{val} \approx 0.9594 \qquad \text{and} \qquad f'(0.5) = y_3.\,\mathrm{deriv} \approx 0.3639.$$

Remark 13.1 Suppose $F : \mathbb{R}^n \to \mathbb{R}^m$.

1. The cost of computing $\mathbf{J}$ by AD is approximately $\omega(\mathbf{J}) = \min(m, n)\omega(F)$, where $\omega(F)$ is the cost of computing F.
2. Also $\omega(\nabla f) \sim \alpha \cdot \omega(f)$, for $\alpha \in [1, 3]$.
3. The cost of a Hessian is typically $\omega(\mathbf{H}) \sim n\omega(f)$.

Sparse Jacobians

For a sparse Jacobian with a structure such as

$$\mathbf{J}(\boldsymbol{x}) = \begin{pmatrix} \times & \times & 0 \\ \times & 0 & \times \\ 0 & \times & \times \end{pmatrix}$$

we can sometimes choose directions that recover multiple entries at once, especially when forward products $\mathbf{J}\boldsymbol{d}$ and reverse products $\mathbf{J}^\top \boldsymbol{d}$ are both available. For instance,

$$\boldsymbol{d}_1 = \begin{pmatrix} 1 \\ 0 \\ 1 \end{pmatrix} \qquad \text{and} \qquad \boldsymbol{d}_2 = \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix}$$

can be more efficient than using every standard basis vector separately when the relevant entries can be disentangled from the resulting products. The computational effort saved can be put towards much more interesting activities.

Lecture 14 — Thursday, October 09, 2014

Finite differences approximate

$$\mathbf{J}(\boldsymbol{x})\boldsymbol{d} \approx \frac{F(\boldsymbol{x} + \alpha \boldsymbol{d}) - F(\boldsymbol{x})}{\alpha},$$

whereas automatic differentiation computes $\mathbf{J}(\boldsymbol{x})\boldsymbol{d}$ and $\mathbf{J}(\boldsymbol{x})^\top \boldsymbol{d}$ to machine precision.

For a larger example of a sparse Jacobian, suppose

$$\mathbf{J} = \begin{pmatrix} \times & \times & 0 & 0 & 0 \\ \times & 0 & \times & 0 & 0 \\ 0 & \times & \times & 0 & 0 \\ 0 & 0 & \times & \times & 0 \\ 0 & 0 & 0 & \times & \times \end{pmatrix}.$$

A naive finite-difference method uses all five standard basis directions. The sparsity pattern allows fewer grouped directions, such as

$$\boldsymbol{d}_1 = \begin{pmatrix} 1 \\ 0 \\ 0 \\ 1 \\ 0 \end{pmatrix}, \qquad \boldsymbol{d}_2 = \begin{pmatrix} 0 \\ 1 \\ 0 \\ 0 \\ 1 \end{pmatrix}, \qquad \text{and} \qquad \boldsymbol{d}_3 = \begin{pmatrix} 0 \\ 0 \\ 1 \\ 0 \\ 0 \end{pmatrix}.$$

The main message is that sparsity and structure can substantially reduce the cost of derivative computations.

3. Unconstrained Optimization and Nonlinear Systems

Quadratic Models and First-Order Equations

We study nonlinear systems

$$F(\boldsymbol{x}) = \mathbf{0}, \quad F : \mathbb{R}^n \to \mathbb{R}^n,$$

and unconstrained minimization problems

$$\min_{\boldsymbol{x}} f(\boldsymbol{x}), \quad f : \mathbb{R}^n \to \mathbb{R}.$$

A linear objective (or "trivial" function) $f(\boldsymbol{x}) = \boldsymbol{c}^\top \boldsymbol{x}$ with $\boldsymbol{c} \neq \mathbf{0}_n$ is unbounded below. Indeed, if $\bar{\boldsymbol{x}} = -\alpha \boldsymbol{c}$ with $\alpha > 0$, then

$$f(\bar{\boldsymbol{x}}) = \boldsymbol{c}^\top(-\alpha \boldsymbol{c}) = -\alpha \|\boldsymbol{c}\|^2 \to -\infty$$

as $\alpha \to \infty$.

The next simplest model is a quadratic function (or "easy" function)

$$f(\boldsymbol{x}) = \boldsymbol{g}^\top \boldsymbol{x} + \frac{1}{2}\boldsymbol{x}^\top \mathbf{H}\boldsymbol{x},$$

where we may assume $\mathbf{H} = \mathbf{H}^\top$. If $\mathbf{H}$ is not symmetric, then

$$\boldsymbol{x}^\top \mathbf{H}\boldsymbol{x} = \frac{1}{2}\big(\boldsymbol{x}^\top \mathbf{H}\boldsymbol{x} + \boldsymbol{x}^\top \mathbf{H}^\top \boldsymbol{x}\big) = \frac{1}{2}\boldsymbol{x}^\top (\mathbf{H} + \mathbf{H}^\top)\boldsymbol{x},$$

and $(\mathbf{H} + \mathbf{H}^\top)/2$ is symmetric. If $\boldsymbol{x}^*$ is a minimizer, then

$$\nabla f(\boldsymbol{x}^*) = \boldsymbol{g} + \mathbf{H}\boldsymbol{x}^* = \mathbf{0}_n \implies \boldsymbol{x}^* = -\mathbf{H}^{-1}\boldsymbol{g}$$

is the natural candidate when $\mathbf{H}$ is nonsingular.

Proposition 14.1 For the easy function

$$f(\boldsymbol{x}) = \boldsymbol{g}^\top \boldsymbol{x} + \frac{1}{2}\boldsymbol{x}^\top \mathbf{H}\boldsymbol{x},$$

the following hold:

1. If $\mathbf{H}$ is symmetric positive definite, then there is a unique global minimizer $\boldsymbol{x}^* = -\mathbf{H}^{-1}\boldsymbol{g}$.
2. If $\mathbf{H}$ is symmetric but not positive semidefinite, then f is unbounded below.
3. If $\mathbf{H}$ is symmetric positive semidefinite and singular, then either f is unbounded below or f has infinitely many minimizers.

Proof. For (1.), assume $\mathbf{H}$ is symmetric positive definite. Then $\mathbf{H}$ is nonsingular, so the first-order condition

$$\nabla f(\boldsymbol{x}) = \boldsymbol{g} + \mathbf{H}\boldsymbol{x} = \mathbf{0}_n$$

has the unique solution $\boldsymbol{x}^* = -\mathbf{H}^{-1}\boldsymbol{g}$. For any $\boldsymbol{x}$,

$$f(\boldsymbol{x}) - f(\boldsymbol{x}^*) = \frac{1}{2}(\boldsymbol{x} - \boldsymbol{x}^*)^\top \mathbf{H}(\boldsymbol{x} - \boldsymbol{x}^*).$$

Because $\mathbf{H}$ is positive definite, the right-hand side is strictly positive whenever $\boldsymbol{x} \neq \boldsymbol{x}^*$. Hence $\boldsymbol{x}^*$ is the unique global minimizer.

For (2.), assume $\mathbf{H}$ is symmetric but not positive semidefinite. Then there exists $\boldsymbol{z} \neq \mathbf{0}_n$ with

$\boldsymbol{z}^\top \mathbf{H}\boldsymbol{z} < 0$. Replacing $\boldsymbol{z}$ by $-\boldsymbol{z}$ if needed, we may assume $\boldsymbol{g}^\top \boldsymbol{z} \leq 0$. Along the ray $\boldsymbol{x} = \alpha \boldsymbol{z}$,

$$f(\alpha \boldsymbol{z}) = \alpha \boldsymbol{g}^\top \boldsymbol{z} + \frac{1}{2}\alpha^2 \boldsymbol{z}^\top \mathbf{H}\boldsymbol{z}.$$

The quadratic coefficient is negative, so $f(\alpha \boldsymbol{z}) \to -\infty$ as $\alpha \to \infty$. Therefore f is unbounded below.

For (3.), assume $\mathbf{H}$ is symmetric positive semidefinite and singular.

If $\boldsymbol{g} \notin \text{Range}(\mathbf{H})$, then, since $\mathbf{H}$ is symmetric,

$$\text{Range}(\mathbf{H}) = (\ker(\mathbf{H}))^\perp.$$

So there exists $\boldsymbol{z} \in \ker(\mathbf{H})$ with $\boldsymbol{g}^\top \boldsymbol{z} \neq 0$. For any $\boldsymbol{x}$,

$$f(\boldsymbol{x} + \alpha \boldsymbol{z}) = f(\boldsymbol{x}) + \alpha \boldsymbol{g}^\top \boldsymbol{z}$$

because $\mathbf{H}\boldsymbol{z} = \mathbf{0}_n$ eliminates the mixed and quadratic terms in α. Choosing the sign of α so that $\alpha \boldsymbol{g}^\top \boldsymbol{z} \to -\infty$ shows that f is unbounded below.

If $\boldsymbol{g} \in \text{Range}(\mathbf{H})$, choose $\boldsymbol{x}^*$ such that $\mathbf{H}\boldsymbol{x}^* = -\boldsymbol{g}$. Then, for any $\boldsymbol{x}$,

$$f(\boldsymbol{x}) - f(\boldsymbol{x}^*) = \frac{1}{2}(\boldsymbol{x} - \boldsymbol{x}^*)^\top \mathbf{H}(\boldsymbol{x} - \boldsymbol{x}^*) \geq 0,$$

so $\boldsymbol{x}^*$ is a global minimizer. Since $\mathbf{H}$ is singular, there exists nonzero $\boldsymbol{z} \in \ker(\mathbf{H})$. Also,

$$\boldsymbol{g}^\top \boldsymbol{z} = (-\mathbf{H}\boldsymbol{x}^*)^\top \boldsymbol{z} = -(\boldsymbol{x}^*)^\top \mathbf{H}\boldsymbol{z} = 0.$$

Hence

$$f(\boldsymbol{x}^* + \boldsymbol{z}) = f(\boldsymbol{x}^*) + \boldsymbol{g}^\top \boldsymbol{z} + \frac{1}{2}\boldsymbol{z}^\top \mathbf{H}\boldsymbol{z} = f(\boldsymbol{x}^*),$$

and similarly $f(\boldsymbol{x}^* + \alpha \boldsymbol{z}) = f(\boldsymbol{x}^*)$ for every $\alpha \in \mathbb{R}$. Therefore there are infinitely many minimizers. □

Lecture 15 — Friday, October 10, 2014

Newton's Method

Most methods for nonlinear minimization are iterative. Starting from $\boldsymbol{x}_0$, they construct $\boldsymbol{x}_1, \boldsymbol{x}_2, \ldots$, with the hope that $\boldsymbol{x}_k \to \boldsymbol{x}_*$ as $k \to \infty$. A method is **globally convergent** if it converges from every starting point, and **locally convergent** if it converges whenever $\boldsymbol{x}_0$ is sufficiently close to the desired solution.

To minimize f, we must solve $\nabla f(\boldsymbol{x}) = \mathbf{0}_n$. This motivates studying nonlinear equations first. For $F(\boldsymbol{x}) = \mathbf{0}_n$, the basic idea is to approximate F near $\boldsymbol{x}_k$ by

$$F(\boldsymbol{x}_k + \boldsymbol{s}) \approx F(\boldsymbol{x}_k) + \mathbf{B}_k \boldsymbol{s},$$

where $\mathbf{B}_k$ is $\mathbf{J}(\boldsymbol{x}_k)$ or an approximation to it. Setting this approximation equal to zero gives

$$\mathbf{B}_k \boldsymbol{s} = -F(\boldsymbol{x}_k),$$

and then $\boldsymbol{x}_{k+1} = \boldsymbol{x}_k + \boldsymbol{s}$.

The general **Newton process** starts with $\boldsymbol{x}_0$, an initial matrix $\mathbf{B}_0$, and $k = 0$:

1. Evaluate $F(\boldsymbol{x}_k)$ and choose $\mathbf{B}_k$.
2. Solve $\mathbf{B}_k \boldsymbol{s}_k = -F(\boldsymbol{x}_k)$.
3. Set $\boldsymbol{x}_{k+1} = \boldsymbol{x}_k + \boldsymbol{s}_k$ and increment k.

If $\mathbf{B}_k = \mathbf{J}(\boldsymbol{x}_k)$ exactly, this is **Newton's method**. If $\mathbf{B}_k$ approximates $\mathbf{J}(\boldsymbol{x}_k)$, the method is a **quasi-Newton method**. Newton-type methods can be very fast, but the basic theory is local: convergence is guaranteed only when the starting point is sufficiently close to the solution and the Jacobian approximation remains sufficiently accurate.

Theorem 15.1 Assume that F is continuously differentiable, $F(\boldsymbol{x}_*) = \mathbf{0}_n$, and $\mathbf{J}(\boldsymbol{x}_*)$ is nonsingular. Generate $\{\boldsymbol{x}_k\}$ by the Newton process. Then there are positive constants δ_1 and δ_2 such that if $\|\boldsymbol{x}_0 - \boldsymbol{x}_*\| \leq \delta_1$ and $\|\mathbf{B}_k - \mathbf{J}(\boldsymbol{x}_k)\| \leq \delta_2$ for all k, then $\{\boldsymbol{x}_k\}$ converges linearly to $\boldsymbol{x}_*$.

Lecture 16 — Tuesday, October 14, 2014

When $\mathbf{B}_k = \mathbf{J}(\boldsymbol{x}_k)$, the update can be written

$$\boldsymbol{x}_{k+1} = \boldsymbol{x}_k - [\mathbf{J}(\boldsymbol{x}_k)]^{-1} F(\boldsymbol{x}_k).$$

When $\mathbf{B}_k \approx \mathbf{J}(\boldsymbol{x}_k)$, the same process gives a quasi-Newton method.

Convergence Rates

1. The sequence $\{\boldsymbol{x}_k\}$ converges **linearly** to $\boldsymbol{x}_*$ if there are $\overline{k}$ and $c \in (0,1)$ such that

   $$\|\boldsymbol{x}_{k+1} - \boldsymbol{x}_*\| \leq c\|\boldsymbol{x}_k - \boldsymbol{x}_*\|$$

   for all $k \geq \overline{k}$. For example, we might have $\text{err}_{k+1} \approx 0.1\ \text{err}_k$.

2. The sequence converges **superlinearly** if

   $$\frac{\|\boldsymbol{x}_{k+1} - \boldsymbol{x}_*\|}{\|\boldsymbol{x}_k - \boldsymbol{x}_*\|} \to 0$$

   as $k \to \infty$, whenever the denominator is nonzero.

3. The sequence converges **quadratically** if there are $\overline{k}$ and $c > 0$ such that

   $$\|\boldsymbol{x}_{k+1} - \boldsymbol{x}_*\| \leq c\|\boldsymbol{x}_k - \boldsymbol{x}_*\|^2$$

   for all $k \geq \overline{k}$.

For example,

$$\{\|\boldsymbol{x}_k - \boldsymbol{x}_*\|\}_{k=1}^{\infty} = \{2^{-k}\}_{k=1}^{\infty}$$

converges linearly, while a typical quadratic pattern looks like $\{2^{-2^k}\}_{k=1}^{\infty}$. Under standard smoothness and nonsingularity assumptions, Newton's method converges quadratically near the solution.

Example 16.1 To compute $\sqrt{a}$ for $a > 0$, solve $F(\boldsymbol{x}) = x^2 - a = 0$. Newton's method gives

$$x_{k+1} = x_k - \frac{x_k^2 - a}{2x_k}.$$

Then

$$x_{k+1} - \sqrt{a} = \frac{1}{2x_k}\left(x_k - \sqrt{a}\right)^2.$$

Thus the error satisfies

$$\left|x_{k+1} - \sqrt{a}\right| = \frac{1}{2|x_k|}\left|x_k - \sqrt{a}\right|^2,$$

which exhibits quadratic convergence when x_k stays near $\sqrt{a}$.

For minimization, the equation to solve is $\nabla f(\boldsymbol{x}) = 0$. This leads to the standard optimality conditions:

Theorem 16.2 (Standard optimality conditions)

1. **First-order necessary condition**: if f is continuously differentiable and $\boldsymbol{x}_*$ is a local minimizer, then $\nabla f(\boldsymbol{x}_*) = \mathbf{0}_n$.
2. **Second-order necessary condition**: if f is twice continuously differentiable and $\boldsymbol{x}_*$ is a local minimizer, then $\nabla f(\boldsymbol{x}_*) = \mathbf{0}_n$ and $\nabla^2 f(\boldsymbol{x}_*)$ is positive semidefinite.
3. **Second-order sufficient condition**: If f is twice continuously differentiable, $\nabla f(\boldsymbol{x}_*) = \mathbf{0}_n$, and $\nabla^2 f(\boldsymbol{x}_*)$ is symmetric positive definite, then $\boldsymbol{x}_*$ is a strict local minimizer.

Editorial Note Lectures 17 and 18 were focused on a review of Assignment 1.

Lecture 19 — Tuesday, October 21, 2014

The second-order sufficient condition says that if $\nabla f(\boldsymbol{x}_*) = \mathbf{0}_n$ and $\nabla^2 f(\boldsymbol{x}_*)$ is symmetric positive definite, then $\boldsymbol{x}_*$ is a strict local minimizer.

Descent Direction Methods

We say that $\boldsymbol{d}$ is a **descent direction** of f at $\bar{\boldsymbol{x}}$ if there is $\bar{\alpha} > 0$ such that

$$f(\bar{\boldsymbol{x}} + \alpha \boldsymbol{d}) < f(\bar{\boldsymbol{x}})$$

for all $0 < \alpha < \bar{\alpha}$.

If $\nabla f(\bar{\boldsymbol{x}})^\top \boldsymbol{d} < 0$, then $\boldsymbol{d}$ is a descent direction. Indeed, Taylor's theorem gives

$$f(\bar{\boldsymbol{x}} + \alpha \boldsymbol{d}) = f(\bar{\boldsymbol{x}}) + \alpha \nabla f(\bar{\boldsymbol{x}})^\top \boldsymbol{d} + o(\alpha) = f(\bar{\boldsymbol{x}}) + \alpha \left(\nabla f(\bar{\boldsymbol{x}})^\top \boldsymbol{d} + \frac{o(\alpha)}{\alpha} \right),$$

which is less than $f(\bar{\boldsymbol{x}})$ for all sufficiently small positive α.

The generic **descent direction method** is as follows:

1. Construct a descent direction $\boldsymbol{d}_k$ with $\nabla f(\boldsymbol{x}_k)^\top \boldsymbol{d}_k < 0$.
2. Choose a step length, often approximately, by minimizing along $\boldsymbol{d}_k$:
$$\alpha_k \approx \arg\min_{\alpha > 0} f(\boldsymbol{x}_k + \alpha \boldsymbol{d}_k).$$
3. Set $\boldsymbol{x}_{k+1} = \boldsymbol{x}_k + \alpha_k \boldsymbol{d}_k$.

A common choice of $\boldsymbol{d}_k$ is the steepest descent direction $\boldsymbol{d}_k = -\nabla f(\boldsymbol{x}_k)$, but this can be hard to control (imagine dropping off the roof of MC in order to get to the ground floor). Another common choice is an approximate Newton direction obtained by solving

$$\mathbf{H}_k \boldsymbol{d}_k = -\nabla f(\boldsymbol{x}_k),$$

where $\mathbf{H}_k \approx \nabla^2 f(\boldsymbol{x}_k)$. In a quasi-Newton method, $\mathbf{H}_{k+1}$ is updated using $\mathbf{H}_k$, the step $\boldsymbol{s}_k = \boldsymbol{x}_{k+1} - \boldsymbol{x}_k$, and the gradient difference $\boldsymbol{y}_k = \nabla f(\boldsymbol{x}_{k+1}) - \nabla f(\boldsymbol{x}_k)$. Low-rank updates let us incorporate this new information without rebuilding the approximation from scratch. A general rank-two update takes the form

$$\mathbf{H}_{k+1} = \mathbf{H}_k + \boldsymbol{a}\boldsymbol{b}^\top + \boldsymbol{c}\boldsymbol{d}^\top.$$

A famous example is the **BFGS** (**Broyden–Fletcher–Goldfarb–Shanno**) method:

$$\mathbf{H}_{k+1} = \mathbf{H}_k + \frac{\boldsymbol{y}_k \boldsymbol{y}_k^\top}{\boldsymbol{y}_k^\top \boldsymbol{s}_k} - \frac{\mathbf{H}_k \boldsymbol{s}_k \boldsymbol{s}_k^\top \mathbf{H}_k}{\boldsymbol{s}_k^\top \mathbf{H}_k \boldsymbol{s}_k}.$$

If $\mathbf{H}_k$ is positive definite and the curvature condition $\boldsymbol{y}_k^\top \boldsymbol{s}_k > 0$ holds, the BFGS update preserves positive definiteness. Under appropriate additional conditions, it also gives superlinear convergence. As a bonus, it provides an explicit formula for $\mathbf{H}_{k+1}^{-1}$.

Theorem 19.1 If $\mathbf{H}_k$ is symmetric positive definite and $\mathbf{H}_k \boldsymbol{d}_k = -\nabla f(\boldsymbol{x}_k)$, then $\boldsymbol{d}_k$ is a descent direction for f at $\boldsymbol{x}_k$.

Proof. Since $\boldsymbol{d}_k = -\mathbf{H}_k^{-1} \nabla f(\boldsymbol{x}_k)$,

$$\nabla f(\boldsymbol{x}_k)^\top \boldsymbol{d}_k = -\nabla f(\boldsymbol{x}_k)^\top \mathbf{H}_k^{-1} \nabla f(\boldsymbol{x}_k).$$

Because $\mathbf{H}_k^{-1}$ is symmetric positive definite, the right-hand side is strictly negative unless $\nabla f(\boldsymbol{x}_k) = \mathbf{0}_n$. Thus $\boldsymbol{d}_k$ is a descent direction whenever $\boldsymbol{x}_k$ is not stationary. □

Lecture 20 — Tuesday, October 28, 2014

The general descent framework will reappear in the trust region method, where the step is controlled by a local model rather than by a line search alone.

Trust Region Methods

Recall that Taylor's theorem gives

$$f(\boldsymbol{x}_k + \boldsymbol{s}) = f(\boldsymbol{x}_k) + \nabla f(\boldsymbol{x}_k)^\top \boldsymbol{s} + \frac{1}{2} \boldsymbol{s}^\top \mathbf{H}(\boldsymbol{x}_k) \boldsymbol{s} + o(\|\boldsymbol{s}\|_2^2).$$

The **trust region method** minimizes this quadratic model inside a ball of radius Δ_k:

$$\min \left\{ \nabla f(\boldsymbol{x}_k)^\top \boldsymbol{s} + \frac{1}{2} \boldsymbol{s}^\top \mathbf{H}(\boldsymbol{x}_k) \boldsymbol{s} \mid \|\boldsymbol{s}\|_2 \le \Delta_k \right\}.$$

The vector $\boldsymbol{s}$ is called the **trial step** for Δ_k. A typical trust region iteration solves this subproblem approximately, accepts $\boldsymbol{x}_{k+1} = \boldsymbol{x}_k + \boldsymbol{s}$ if the actual decrease is sufficient, and adjusts Δ_k according to how well the model predicts the actual objective decrease.

The usual ratio compares actual reduction to predicted reduction:

$$\rho_k = \frac{f(\boldsymbol{x}_k + \boldsymbol{s}) - f(\boldsymbol{x}_k)}{\nabla f(\boldsymbol{x}_k)^\top \boldsymbol{s} + \frac{1}{2} \boldsymbol{s}^\top \mathbf{H}(\boldsymbol{x}_k) \boldsymbol{s}}.$$

If the higher-order term is negligible, then $\rho_k \approx 1$. In practice, we may enlarge the region when

$\rho_k > 3/4$, shrink it when $\rho_k < 1/4$, and otherwise leave it unchanged.

Under standard assumptions, a well-designed trust region method converges to a stationary point; it provides no blanket guarantee of finding a global minimizer (go hire a guy from IBM for that). Near a nondegenerate local minimizer, methods that eventually take the full Newton step can converge quadratically.

The trust region subproblem of finding the constrained minimizer is subtle, but efficient methods exist. If there were no trust region constraint, the model

$$\boldsymbol{g}^\top \boldsymbol{s} + \frac{1}{2}\boldsymbol{s}^\top \mathbf{H}\boldsymbol{s}$$

would have the minimizer $\boldsymbol{s}_* = -\mathbf{H}^{-1}\boldsymbol{g}$ when $\mathbf{H}$ is symmetric positive definite. With the constraint present, the solution is characterized by a parameter $\lambda_k \geq 0$ satisfying

$$\begin{cases} (\mathbf{H}(\boldsymbol{x}_k) + \lambda_k \mathbf{I})\boldsymbol{s} = -\nabla f(\boldsymbol{x}_k), \\ \lambda_k(\Delta_k - \|\boldsymbol{s}\|_2) = 0. \end{cases}$$

If the unconstrained step satisfies $\|\boldsymbol{s}\|_2 \leq \Delta_k$, then $\lambda_k = 0$. Otherwise, the solution lies on the boundary $\|\boldsymbol{s}\|_2 = \Delta_k$. In general, λ_k is chosen so that $\mathbf{H}(\boldsymbol{x}_k) + \lambda_k \mathbf{I}$ is positive semidefinite; equivalently, λ_k must be at least $-\lambda_{\min}$ when $\lambda_{\min}$ is the smallest eigenvalue of $\mathbf{H}(\boldsymbol{x}_k)$.

Lecture 21 — Thursday, October 30, 2014

The practical stopping conditions for an iterative method are usually based on small first-order residuals or small steps; for example, we might stop when

$$\|\nabla f(\boldsymbol{x}_k)\| \leq \mathrm{tol}_1 \qquad \text{or} \qquad \|\boldsymbol{x}_{k+1} - \boldsymbol{x}_k\| \leq \mathrm{tol}_2\,.$$

For nonlinear least squares,

$$f(\boldsymbol{x}) = \frac{1}{2}\|F(\boldsymbol{x})\|_2^2,$$

and we have

$$\nabla f(\boldsymbol{x}) = \mathbf{J}(\boldsymbol{x})^\top F(\boldsymbol{x}) \qquad \text{and} \qquad \nabla^2 f(\boldsymbol{x}) = \mathbf{J}(\boldsymbol{x})^\top \mathbf{J}(\boldsymbol{x}) + \sum_{i=1}^{m} F_i(\boldsymbol{x})\nabla^2 F_i(\boldsymbol{x}).$$

For a descent direction method, we could choose $\boldsymbol{d}_k = -\nabla f(\boldsymbol{x}_k)$, but this usually moves too quickly and we risk losing control. A better choice is the Newton step $\mathbf{H}(\boldsymbol{x}_k)\boldsymbol{d}_k = -\nabla f(\boldsymbol{x}_k)$; however, computing $\mathbf{H}(\boldsymbol{x}_k)$ is expensive, so we usually opt for an approximation instead. This explains the Gauss–Newton approximation

$$\mathbf{J}(\boldsymbol{x}_k)^\top \mathbf{J}(\boldsymbol{x}_k)\boldsymbol{d}_k \approx -\nabla f(\boldsymbol{x}_k)$$

and the Levenberg–Marquardt variant

$$\big(\mathbf{J}(\boldsymbol{x}_k)^\top \mathbf{J}(\boldsymbol{x}_k) + \lambda \mathbf{I}\big)\boldsymbol{d}_k = -\nabla f(\boldsymbol{x}_k).$$

The most expensive part of the trust region method is the trust region subproblem. If we use an approximation, the optimization problem can also be written in least-squares form as

$$\min\left\{\|\mathbf{J}(\boldsymbol{x}_k)\boldsymbol{s} + F(\boldsymbol{x}_k)\|_2^2 \mid \|\boldsymbol{s}\| \le \Delta_k\right\}.$$

Lecture 22 — Friday, October 31, 2014

4. Constrained Optimization

So far we have considered unconstrained problems of the form $\min f(\boldsymbol{x})$, where $f : \mathbb{R}^n \to \mathbb{R}$. We now introduce constraints.

Equality-Constrained Optimization

Instead of searching over all of $\mathbb{R}^n$, we search over a feasible set such as an affine subspace.

As a finance example, suppose Eva wants to invest in 12 stocks $S_1, \ldots, S_{12}$ with portfolio weights $x_1, \ldots, x_{12}$ satisfying the budget constraint

$$\sum_{i=1}^{12} x_i = 1.$$

Suppose 25% of the investment must go to the technology stocks $S_1, \ldots, S_4$, and 20% must go to

the energy stocks S_{10}, S_{11}, S_{12}. If the desired portfolio return is r_p, then

$$\sum_{i=1}^{12} x_i \bar{r}_i = r_p,$$

where $\bar{r}_i$ is the expected return of stock i.

Suppose the returns are estimated from 100 observations collected in the data matrix

$$\mathbf{R} = \begin{pmatrix} r_1^1 & \dots & r_{12}^1 \\ \vdots & \ddots & \vdots \\ r_1^{100} & \dots & r_{12}^{100} \end{pmatrix} \in \mathbb{R}^{100\times 12}.$$

Then

$$\bar{r}_i = \frac{1}{100}\sum_{j=1}^{100} r_i^j.$$

Let $\bar{\boldsymbol{r}} = (\bar{r}_1, \dots, \bar{r}_{12})^\top$, and let $\bar{\mathbf{R}} \in \mathbb{R}^{100\times 12}$ denote the matrix whose every row is $\bar{\boldsymbol{r}}^\top$. The goal is to minimize portfolio risk:

$$\min_{\boldsymbol{x}} \text{Risk}(\boldsymbol{x}) := \mathbb{E}\left[(\bar{\boldsymbol{r}}^\top \boldsymbol{x} - \boldsymbol{r}^\top \boldsymbol{x})^2\right] = \frac{1}{100}\sum_{i=1}^{100}\left[\bar{\boldsymbol{r}}^\top \boldsymbol{x} - (\boldsymbol{r}^i)^\top \boldsymbol{x}\right]^2.$$

In matrix form the objective function is

$$\frac{1}{100}\|\bar{\mathbf{R}}\boldsymbol{x} - \mathbf{R}\boldsymbol{x}\|_2^2 = \frac{1}{100}\|\mathbf{A}_1\boldsymbol{x}\|_2^2,$$

where $\mathbf{A}_1 = \bar{\mathbf{R}} - \mathbf{R}$. The complete constrained problem is therefore to minimize $\|\mathbf{A}_1\boldsymbol{x}\|_2^2$ subject to

$$\sum_{i=1}^{12} x_i = 1, \qquad (1,1,1,1,0,\dots,0)\boldsymbol{x} = 0.25, \qquad (0,\dots,0,1,1,1)\boldsymbol{x} = 0.20, \qquad \text{and} \qquad \bar{\boldsymbol{r}}^\top \boldsymbol{x} = r_p.$$

Let

$$\mathbf{A}_2 = \begin{pmatrix} 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 1 \\ \bar{r}_1 & \bar{r}_2 & \bar{r}_3 & \bar{r}_4 & \bar{r}_5 & \bar{r}_6 & \bar{r}_7 & \bar{r}_8 & \bar{r}_9 & \bar{r}_{10} & \bar{r}_{11} & \bar{r}_{12} \end{pmatrix} \qquad \text{and} \qquad \boldsymbol{b} = \begin{pmatrix} 1 \\ 0.25 \\ 0.20 \\ r_p \end{pmatrix}.$$

Then the problem becomes

$$\min \boldsymbol{x}^\top \mathbf{A}_1^\top \mathbf{A}_1 \boldsymbol{x} \quad \text{subject to} \quad \mathbf{A}_2 \boldsymbol{x} = \boldsymbol{b},$$

which is a quadratic problem with linear equality constraints.

Null Space Approach and Optimality Conditions

Consider the quadratic problem

$$\min \left\{ \boldsymbol{g}^\top \boldsymbol{x} + \frac{1}{2} \boldsymbol{x}^\top \mathbf{H} \boldsymbol{x} \mid \mathbf{A}\boldsymbol{x} = \boldsymbol{b} \right\}.$$

Assume $\mathbf{Z}$ is a basis for Null($\mathbf{A}$) and $\boldsymbol{x}_0$ is a known feasible point. Then every feasible point can be written as $\boldsymbol{x} = \boldsymbol{x}_0 + \mathbf{Z}\boldsymbol{w}$. Substitution gives the unconstrained problem $\min_{\boldsymbol{w}} \bar{f}(\boldsymbol{w})$, where

$$\begin{aligned} \bar{f}(\boldsymbol{w}) &= \boldsymbol{g}^\top (\boldsymbol{x}_0 + \mathbf{Z}\boldsymbol{w}) + \frac{1}{2} (\boldsymbol{x}_0 + \mathbf{Z}\boldsymbol{w})^\top \mathbf{H} (\boldsymbol{x}_0 + \mathbf{Z}\boldsymbol{w}) \\ &= \boldsymbol{g}^\top \boldsymbol{x}_0 + \boldsymbol{w}^\top \mathbf{Z}^\top \boldsymbol{g} + \frac{1}{2} \boldsymbol{x}_0^\top \mathbf{H} \boldsymbol{x}_0 + \boldsymbol{w}^\top \mathbf{Z}^\top \mathbf{H} \boldsymbol{x}_0 + \frac{1}{2} \boldsymbol{w}^\top \mathbf{Z}^\top \mathbf{H} \mathbf{Z} \boldsymbol{w}. \end{aligned}$$

The first-order condition is

$$\nabla \bar{f}(\boldsymbol{w}) = \mathbf{Z}^\top \boldsymbol{g} + \mathbf{Z}^\top \mathbf{H} \boldsymbol{x}_0 + \mathbf{Z}^\top \mathbf{H} \mathbf{Z} \boldsymbol{w} = \mathbf{0}_{n-m},$$

so

$$(\mathbf{Z}^\top \mathbf{H} \mathbf{Z}) \boldsymbol{w} = \mathbf{Z}^\top (-\boldsymbol{g} - \mathbf{H} \boldsymbol{x}_0).$$

If $\mathbf{H}$ is symmetric positive definite, then $\mathbf{Z}^\top \mathbf{H} \mathbf{Z}$ is symmetric positive definite on the reduced space, so Cholesky factorization can be used.

The resulting null space algorithm is as follows:

1. Compute $\boldsymbol{x}_0$ and $\mathbf{Z}$, for example by the QR method.
2. Form $\bar{\mathbf{H}} = \mathbf{Z}^\top \mathbf{H} \mathbf{Z}$ and $\bar{\boldsymbol{r}} = \mathbf{Z}^\top (-\boldsymbol{g} - \mathbf{H} \boldsymbol{x}_0)$.
3. Solve $\bar{\mathbf{H}} \boldsymbol{w} = \bar{\boldsymbol{r}}$ by Cholesky factorization.
4. Set $\boldsymbol{x}_* = \boldsymbol{x}_0 + \mathbf{Z}\boldsymbol{w}$.

Lecture 23 — Tuesday, November 04, 2014

Now consider the more general equality-constrained problem

$$\min\{f(\boldsymbol{x}) \mid \mathbf{A}\boldsymbol{x} = \boldsymbol{b}\},$$

where $\mathbf{A} \in \mathbb{R}^{m\times n}$, $m < n$, and $\text{rank}(\mathbf{A}) = m$.

Geometrically, in the two-dimensional case with $\mathbf{A} = \boldsymbol{a}^\top$, the constraint $\boldsymbol{a}^\top\boldsymbol{x} = b$ is a line. At a constrained minimizer, the gradient of f must be orthogonal to the feasible directions and hence parallel to $\boldsymbol{a}$: there can be no gradient component along the feasible line.

Theorem 23.1 If $\bar{\boldsymbol{x}}$ is a minimizer of $\min\{f(\boldsymbol{x}) \mid \mathbf{A}\boldsymbol{x} = \boldsymbol{b}\}$, then $\nabla f(\bar{\boldsymbol{x}}) = \mathbf{A}^\top\boldsymbol{\lambda}$ for some $\boldsymbol{\lambda} \in \mathbb{R}^m$.

Proof. Let the columns of $\mathbf{Z}$ form a basis for $\text{Null}(\mathbf{A})$. If $\nabla f(\bar{\boldsymbol{x}})$ has a nonzero null-space component, write $\nabla f(\bar{\boldsymbol{x}}) = \mathbf{A}^\top\boldsymbol{\lambda}_1 + \mathbf{Z}\boldsymbol{\lambda}_2$ with $\boldsymbol{\lambda}_2 \neq \mathbf{0}_{n-m}$. Taking $\boldsymbol{d} = -\mathbf{Z}\boldsymbol{\lambda}_2$, we have $\mathbf{A}\boldsymbol{d} = \mathbf{0}_m$, so $\bar{\boldsymbol{x}} + \alpha\boldsymbol{d}$ remains feasible for small α. Moreover,

$$f(\bar{\boldsymbol{x}} + \alpha\boldsymbol{d}) = f(\bar{\boldsymbol{x}}) + \alpha\nabla f(\bar{\boldsymbol{x}})^\top\boldsymbol{d} + o(\alpha) = f(\bar{\boldsymbol{x}}) - \alpha\boldsymbol{\lambda}_2^\top\mathbf{Z}^\top\mathbf{Z}\boldsymbol{\lambda}_2 + o(\alpha) < f(\bar{\boldsymbol{x}})$$

for sufficiently small positive α, a contradiction. □

Theorem 23.2 Let $\mathbf{Z}$ be a basis for $\text{Null}(\mathbf{A})$. We have $\nabla f(\bar{\boldsymbol{x}}) = \mathbf{A}^\top\boldsymbol{\lambda}$ for some $\boldsymbol{\lambda}$ if and only if $\mathbf{Z}^\top\nabla f(\bar{\boldsymbol{x}}) = \mathbf{0}_{n-m}$. Equivalently, $\nabla f(\bar{\boldsymbol{x}})$ is orthogonal to $\text{Null}(\mathbf{A})$.

Proof. If $\nabla f(\bar{\boldsymbol{x}}) = \mathbf{A}^\top\boldsymbol{\lambda}$, then $\mathbf{Z}^\top\nabla f(\bar{\boldsymbol{x}}) = \mathbf{Z}^\top\mathbf{A}^\top\boldsymbol{\lambda} = \mathbf{0}_{n-m}$. Conversely, decompose $\nabla f(\bar{\boldsymbol{x}}) = \mathbf{A}^\top\boldsymbol{\lambda} + \mathbf{Z}\boldsymbol{\lambda}_2$. If $\mathbf{Z}^\top\nabla f(\bar{\boldsymbol{x}}) = \mathbf{0}_{n-m}$, then $\mathbf{Z}^\top\mathbf{Z}\boldsymbol{\lambda}_2 = \mathbf{0}_{n-m}$. Since $\mathbf{Z}^\top\mathbf{Z}$ is symmetric positive definite, $\boldsymbol{\lambda}_2 = \mathbf{0}_{n-m}$, so $\nabla f(\bar{\boldsymbol{x}}) = \mathbf{A}^\top\boldsymbol{\lambda}$. □

The null space approach reduces the constrained problem to an unconstrained one. Its optimality conditions are given here:

Theorem 23.3 (Null space approach: optimality conditions)

1. **First-order necessary condition**: if $\boldsymbol{x}_*$ is a local minimizer and $\boldsymbol{x}_* = \boldsymbol{x}_0 + \mathbf{Z}\boldsymbol{w}$, then $\mathbf{Z}^\top \nabla f(\boldsymbol{x}_*) = \mathbf{0}_{n-m}$.
2. **Second-order necessary condition**: if $\boldsymbol{x}_*$ is a local minimizer, then $\mathbf{Z}^\top \nabla f(\boldsymbol{x}_*) = \mathbf{0}_{n-m}$ and $\mathbf{Z}^\top \nabla^2 f(\boldsymbol{x}_*)\mathbf{Z}$ is positive semidefinite.
3. **Second-order sufficiency condition**: if $\mathbf{Z}^\top \nabla f(\boldsymbol{x}_*) = \mathbf{0}_{n-m}$ and $\mathbf{Z}^\top \nabla^2 f(\boldsymbol{x}_*)\mathbf{Z}$ is symmetric positive definite, then $\boldsymbol{x}_*$ is a local minimizer.

These conditions motivate the full-space approach.

Lecture 24 — Thursday, November 06, 2014

Full Space and Range-Space Approaches

Consider the equality-constrained quadratic model

$$\min \left\{ \boldsymbol{g}^\top \boldsymbol{x} + \frac{1}{2}\boldsymbol{x}^\top \mathbf{H}\boldsymbol{x} \mid \mathbf{A}\boldsymbol{x} = \boldsymbol{b} \right\},$$

where $\mathbf{A} \in \mathbb{R}^{m \times n}$.

In the **full-space approach**, we keep both primal and dual variables. First-order stationarity implies that there exists $\boldsymbol{\lambda} \in \mathbb{R}^m$ such that

$$\mathbf{H}\boldsymbol{x} + \mathbf{A}^\top \boldsymbol{\lambda} = -\boldsymbol{g}.$$

Together with feasibility, this gives the system

$$\begin{pmatrix} \mathbf{H} & \mathbf{A}^\top \\ \mathbf{A} & \mathbf{0}_{m\times m} \end{pmatrix} \begin{pmatrix} \boldsymbol{x} \\ \boldsymbol{\lambda} \end{pmatrix} = \begin{pmatrix} -\boldsymbol{g} \\ \boldsymbol{b} \end{pmatrix}.$$

This formulation is direct, but the linear system has size $(n+m)$, so a dense solve costs about $O((n+m)^3)$. Also, a solution is a stationary point and may fail to be a minimizer if second-order conditions do not hold.

In the **range-space approach**, assume $\mathbf{H}$ is symmetric positive definite and eliminate $\boldsymbol{x}$. From

$$\boldsymbol{x} = -\mathbf{H}^{-1}(\boldsymbol{g} + \mathbf{A}^\top \boldsymbol{\lambda}),$$

substitution into $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$ yields the reduced multiplier system

$$(\mathbf{A}\mathbf{H}^{-1}\mathbf{A}^\top)\boldsymbol{\lambda} = -\boldsymbol{b} - \mathbf{A}\mathbf{H}^{-1}\boldsymbol{g}.$$

Then

$$\boldsymbol{x} = -\mathbf{H}^{-1}\boldsymbol{g} - \mathbf{H}^{-1}\mathbf{A}^\top \boldsymbol{\lambda}.$$

Because the reduced system is only $m \times m$, this approach is attractive when $m \ll n$.

A practical implementation is:

1. Compute the Cholesky factorization $\mathbf{H} = \mathbf{L}\mathbf{L}^\top$ and define $\tilde{\mathbf{A}} = \mathbf{A}\mathbf{L}^{-\top}$.
2. Solve $\mathbf{L}\boldsymbol{u} = -\boldsymbol{g}$ and $\mathbf{L}^\top \boldsymbol{v} = \boldsymbol{u}$, so $\boldsymbol{v} = -\mathbf{H}^{-1}\boldsymbol{g}$.
3. Form $\tilde{\boldsymbol{b}} = \mathbf{A}\boldsymbol{v} - \boldsymbol{b}$.
4. Solve $(\tilde{\mathbf{A}}\tilde{\mathbf{A}}^\top)\boldsymbol{w} = \tilde{\boldsymbol{b}}$.
5. Solve $\mathbf{L}\boldsymbol{y} = \mathbf{A}^\top \boldsymbol{w}$ and $\mathbf{L}^\top \boldsymbol{t} = \boldsymbol{y}$.
6. Set $\boldsymbol{x} = \boldsymbol{v} - \boldsymbol{t}$.

Lecture 25 — Friday, November 07, 2014

Nonlinear Objective Function with Linear Equalities

Unconstrained minimization ideas can be adapted to linear equality-constrained problems. In particular, the subspace trust region framework extends naturally. The key object is the reduced Hessian $\mathbf{Z}^\top \mathbf{H}\mathbf{Z}$, which is positive definite in a neighborhood of a local constrained minimizer. In that neighborhood, the same methods used for unconstrained problems can be applied to compute constrained Newton steps.

At feasible points far from a local minimizer, the reduced Hessian may be indefinite. In that setting, a trust-region strategy is still effective and can be adapted directly to the constrained case. In MATLAB, `fmincon` implements a subspace trust-region method.

Recall Eva's portfolio model:

$$\min_{\boldsymbol{x}\in\mathbb{R}^{12}} \|\mathbf{A}_1\boldsymbol{x}\|_2^2 \quad \text{subject to} \quad \mathbf{A}_2\boldsymbol{x} = \boldsymbol{b},$$

where $\mathbf{A}_1 = \bar{\mathbf{R}} - \mathbf{R}$,

$$\mathbf{A}_2 = \begin{pmatrix} 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 1 \\ \bar{r}_1 & \bar{r}_2 & \bar{r}_3 & \bar{r}_4 & \bar{r}_5 & \bar{r}_6 & \bar{r}_7 & \bar{r}_8 & \bar{r}_9 & \bar{r}_{10} & \bar{r}_{11} & \bar{r}_{12} \end{pmatrix} \quad \text{and} \quad \boldsymbol{b} = \begin{pmatrix} 1 \\ 0.25 \\ 0.20 \\ r_p \end{pmatrix}.$$

To parameterize all feasible points, compute a QR factorization of $\mathbf{A}_2^\top$:

$$\mathbf{A}_2^\top = \mathbf{Q}_2 \begin{pmatrix} \mathbf{R}_2 \\ \mathbf{0}_{8\times 4} \end{pmatrix} = (\mathbf{Y}_2, \mathbf{Z}_2) \begin{pmatrix} \mathbf{R}_2 \\ \mathbf{0}_{8\times 4} \end{pmatrix} = \mathbf{Y}_2\mathbf{R}_2,$$

where $\mathbf{Y}_2 \in \mathbb{R}^{12\times 4}$ and $\mathbf{Z}_2 \in \mathbb{R}^{12\times 8}$. Then all solutions of $\mathbf{A}_2\boldsymbol{x} = \boldsymbol{b}$ are

$$\boldsymbol{x} = \boldsymbol{x}_0 + \mathbf{Z}_2\boldsymbol{w}, \qquad \boldsymbol{w} \in \mathbb{R}^8, \qquad \text{and} \qquad \boldsymbol{x}_0 = \mathbf{Y}_2\mathbf{R}_2^{-\top}\boldsymbol{b}.$$

Substituting into the objective gives an unconstrained least-squares problem:

$$\min_{\boldsymbol{w}\in\mathbb{R}^8} \|\mathbf{A}_1(\boldsymbol{x}_0 + \mathbf{Z}_2\boldsymbol{w})\|_2^2 = \min_{\boldsymbol{w}\in\mathbb{R}^8} \|\bar{\boldsymbol{b}} + \bar{\mathbf{A}}\boldsymbol{w}\|_2^2,$$

where

$$\bar{\mathbf{A}} = \mathbf{A}_1\mathbf{Z}_2 \in \mathbb{R}^{100\times 8} \qquad \text{and} \qquad \bar{\boldsymbol{b}} = \mathbf{A}_1\boldsymbol{x}_0 \in \mathbb{R}^{100}.$$

Solve this overdetermined system by QR:

$$\bar{\mathbf{A}} = \mathbf{Q} \begin{pmatrix} \bar{\mathbf{R}} \\ \mathbf{0}_{92\times 8} \end{pmatrix} = (\mathbf{Y}, \mathbf{Z}) \begin{pmatrix} \bar{\mathbf{R}} \\ \mathbf{0}_{92\times 8} \end{pmatrix} = \mathbf{Y}\bar{\mathbf{R}},$$

so

$$\boldsymbol{v}_w = -\bar{\mathbf{R}}^{-1}\mathbf{Y}^\top\bar{\boldsymbol{b}}.$$

Hence Eva's optimal strategy is

$$\boldsymbol{x}_* = \boldsymbol{x}_0 + \mathbf{Z}_2\boldsymbol{v}_w = \mathbf{Y}_2\mathbf{R}_2^{-\top}\boldsymbol{b} - \mathbf{Z}_2\bar{\mathbf{R}}^{-1}\mathbf{Y}^\top\bar{\boldsymbol{b}}.$$

The computation can be summarized as follows:

1. Form $\mathbf{A}_1 = \bar{\mathbf{R}} - \mathbf{R} \in \mathbb{R}^{100\times 12}$.
2. Form $\mathbf{A}_2$ and $\boldsymbol{b} = \begin{pmatrix} 1 & 0.25 & 0.20 & r_p \end{pmatrix}^\top$.
3. Compute $\mathbf{A}_2^\top = \mathbf{Q}_2 \begin{pmatrix} \mathbf{R}_2 \\ \mathbf{0}_{8\times 4} \end{pmatrix} = (\mathbf{Y}_2, \mathbf{Z}_2) \begin{pmatrix} \mathbf{R}_2 \\ \mathbf{0}_{8\times 4} \end{pmatrix} = \mathbf{Y}_2\mathbf{R}_2$.
4. Compute $\boldsymbol{x}_0 = \mathbf{Y}_2\mathbf{R}_2^{-\top}\boldsymbol{b}$.
5. Form $\bar{\mathbf{A}} = \mathbf{A}_1\mathbf{Z}_2 \in \mathbb{R}^{100\times 8}$ and $\bar{\boldsymbol{b}} = \mathbf{A}_1\boldsymbol{x}_0 \in \mathbb{R}^{100}$.
6. Compute $\bar{\mathbf{A}} = \mathbf{Q} \begin{pmatrix} \bar{\mathbf{R}} \\ \mathbf{0}_{92\times 8} \end{pmatrix} = (\mathbf{Y}, \mathbf{Z}) \begin{pmatrix} \bar{\mathbf{R}} \\ \mathbf{0}_{92\times 8} \end{pmatrix} = \mathbf{Y}\bar{\mathbf{R}}$.
7. Compute $\boldsymbol{v}_w = -\bar{\mathbf{R}}^{-1}\mathbf{Y}^\top\bar{\boldsymbol{b}}$.
8. Compute $\boldsymbol{x}_* = \boldsymbol{x}_0 + \mathbf{Z}_2\boldsymbol{v}_w = \mathbf{Y}_2\mathbf{R}_2^{-\top}\boldsymbol{b} - \mathbf{Z}_2\bar{\mathbf{R}}^{-1}\mathbf{Y}^\top\bar{\boldsymbol{b}}$.

Lecture 26 — Tuesday, November 11, 2014

Inequality Constraints and Interior Methods

We next allow both equality and inequality constraints:

$$\min_{\boldsymbol{x}}\{f(\boldsymbol{x}) \mid \mathbf{B}\boldsymbol{x} = \boldsymbol{b},\ \mathbf{A}\boldsymbol{x} \le \boldsymbol{d}\}. \tag{$*$}$$

In Eva's investment problem, suppose we add the restrictions

$$x_5 + x_6 + x_7 \le 10\% \qquad \text{and} \qquad x_8 + x_9 \ge 15\%.$$

The second inequality is equivalently $-x_8 - x_9 \le -15\%$. These can be written as

$$\mathbf{A}_3 x \le d, \qquad \mathbf{A}_3 = \begin{pmatrix} 0 & 0 & 0 & 0 & 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & -1 & -1 & 0 & 0 & 0 \end{pmatrix}, \qquad \text{and} \qquad \boldsymbol{d} = \begin{pmatrix} 0.10 \\ -0.15 \end{pmatrix}.$$

Theorem 26.1 Assume $\boldsymbol{x}^*$ is a minimizer of $(*)$. Then there exist multipliers $\boldsymbol{\mu}^*$ and $\boldsymbol{\lambda}^*$ such that

$$\nabla f(\boldsymbol{x}^*) + \mathbf{A}^\top\boldsymbol{\mu}^* + \mathbf{B}^\top\boldsymbol{\lambda}^* = \mathbf{0}_n, \qquad \mathbf{A}\boldsymbol{x}^* \le \boldsymbol{d}, \qquad \text{and} \qquad \mathbf{B}\boldsymbol{x}^* = \boldsymbol{b},$$

$$\mu_i^*(\mathbf{A}_i\boldsymbol{x}^* - d_i) = 0 \quad (i = 1, \ldots, p) \qquad \text{and} \qquad \boldsymbol{\mu}^* \geq \mathbf{0}_p.$$

The equality-constrained condition $\nabla f(\boldsymbol{x}^*) + \mathbf{B}^\top \boldsymbol{\lambda}^* = \mathbf{0}_n$ is the special case with no inequalities.

What does it all mean? The complementarity condition means that each inequality is either inactive, in which case $(\mathbf{A}\boldsymbol{x}^* - \boldsymbol{d})_i < 0$ and $\mu_i^* = 0$, or active, in which case $(\mathbf{A}\boldsymbol{x}^* - \boldsymbol{d})_i = 0$ and the multiplier may be positive.

Example 26.2 For a two-variable example, consider

$$\min(x_1 - 2)^2 + 2(x_2 - 1)^2 \quad \text{subject to} \quad x_1 + 4x_2 \leq 3 \qquad \text{and} \qquad x_1 - x_2 \geq 0.$$

Equivalently,

$$\mathbf{A} = \begin{pmatrix} 1 & 4 \\ -1 & 1 \end{pmatrix}, \qquad \boldsymbol{d} = \begin{pmatrix} 3 \\ 0 \end{pmatrix}, \qquad \text{and} \qquad \mathbf{A}\boldsymbol{x} \leq \boldsymbol{d}.$$

The stationarity condition from Theorem 26.1 is

$$\nabla f(\boldsymbol{x}) + \mathbf{A}^\top \boldsymbol{\mu} = \begin{pmatrix} 2(x_1 - 2) \\ 4(x_2 - 1) \end{pmatrix} + \begin{pmatrix} 1 & -1 \\ 4 & 1 \end{pmatrix} \begin{pmatrix} \mu_1 \\ \mu_2 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \end{pmatrix}.$$

Complementarity gives

$$\mu_1(x_1 + 4x_2 - 3) = 0, \qquad \mu_2(-x_1 + x_2) = 0, \qquad \text{and} \qquad \mu_1, \mu_2 \geq 0.$$

We check the possible active sets:

1. If $\mu_1 = 0$ and $\mu_2 = 0$, then $x_1 = 2$ and $x_2 = 1$. This candidate is infeasible.
2. If $\mu_1 = 0$, then $-x_1 + x_2 = 0$. Then
$$x_1 = \frac{4}{3}, \qquad x_2 = \frac{4}{3}, \qquad \text{and} \qquad \mu_2 = -\frac{4}{3}.$$
This candidate violates the first inequality and has a negative multiplier.
3. If $\mu_2 = 0$, then $3 - x_1 - 4x_2 = 0$. Then
$$x_1 = \frac{5}{3}, \qquad x_2 = \frac{1}{3}, \qquad \text{and} \qquad \mu_1 = \frac{2}{3}.$$

This candidate is feasible and has a nonnegative multiplier! Let's table it for now.

4. If $3 - x_1 - 4x_2 = 0$, then $x_1 - x_2 = 0$. Then

$$x_1 = x_2 = \frac{3}{5}, \qquad \mu_1 = \frac{22}{25}, \qquad \text{and} \qquad \mu_2 = -\frac{48}{25} < 0,$$

so the multiplier condition fails.

Therefore the minimizer is

$$\boldsymbol{x} = \begin{pmatrix} 5/3 \\ 1/3 \end{pmatrix}.$$

In two dimensions this active-set enumeration is feasible; in high dimensions it becomes prohibitively expensive. Avoid it when out doing fieldwork.

Lecture 27 — Thursday, November 13, 2014

The Quadratic Problem

We focus on our beloved quadratic objective function

$$f(\boldsymbol{x}) = \frac{1}{2}\boldsymbol{x}^\top \mathbf{H}\boldsymbol{x} - \boldsymbol{c}^\top \boldsymbol{x}.$$

We are interested in the inequality-constrained problem $\min\{f(\boldsymbol{x}) \mid \mathbf{A}\boldsymbol{x} \leq \boldsymbol{d}\}$. Equality constraints $\mathbf{B}\boldsymbol{x} = \boldsymbol{b}$ can first be handled by the null-space substitution $\boldsymbol{x} = \boldsymbol{x}_0 + \mathbf{Z}\boldsymbol{w}$, leaving an equivalent inequality-constrained problem in $\boldsymbol{w}$.

The conditions from Theorem 26.1 specialized to the quadratic inequality-constrained problem are

$$\mathbf{H}\boldsymbol{x}^* + \mathbf{A}^\top \boldsymbol{\mu}^* - \boldsymbol{c} = \mathbf{0}_n \qquad \text{and} \qquad \mathbf{A}\boldsymbol{x}^* - \boldsymbol{d} \leq \mathbf{0}_p,$$

$$\mu_i^*(\mathbf{A}_i\boldsymbol{x}^* - d_i) = 0 \quad (i = 1, \ldots, p) \qquad \text{and} \qquad \boldsymbol{\mu}^* \geq \mathbf{0}_p.$$

Introduce a slack variable $\boldsymbol{z} = \boldsymbol{d} - \mathbf{A}\boldsymbol{x}$. The conditions become

$$\mathbf{H}\boldsymbol{x}^* + \mathbf{A}^\top \boldsymbol{\mu}^* - \boldsymbol{c} = \mathbf{0}_n$$

$$\mathbf{A}\boldsymbol{x}^* + \boldsymbol{z} - \boldsymbol{d} = \mathbf{0}_p$$
$$\mathbf{Z}\boldsymbol{\mu} = \mathbf{0}_p,$$

where $\mathbf{Z} = \operatorname{diag}(z_1, \ldots, z_p)$ and $\mathbf{D}_\mu = \operatorname{diag}(\mu_1, \ldots, \mu_p)$. Combine these equations into

$$\mathbf{F}(\boldsymbol{x}, \boldsymbol{\mu}, \boldsymbol{z}) = \begin{pmatrix} \mathbf{H}\boldsymbol{x} + \mathbf{A}^\top \boldsymbol{\mu} - \boldsymbol{c} \\ \mathbf{A}\boldsymbol{x} + \boldsymbol{z} - \boldsymbol{d} \\ \mathbf{Z}\mathbf{D}_\mu \mathbf{1} \end{pmatrix} = \mathbf{0}_{n+2p}.$$

We only know one way to (iteratively) solve $\mathbf{F}(\boldsymbol{x}, \boldsymbol{\mu}, \boldsymbol{z}) = \mathbf{0}_{n+2p}$, and that is Newton's method. Generically, we have

$$\begin{pmatrix} \boldsymbol{x}_{k+1} \\ \boldsymbol{\mu}_{k+1} \\ \boldsymbol{z}_{k+1} \end{pmatrix} = \begin{pmatrix} \boldsymbol{x}_k \\ \boldsymbol{\mu}_k \\ \boldsymbol{z}_k \end{pmatrix} - [\mathbf{F}'(\boldsymbol{x}_k, \boldsymbol{\mu}_k, \boldsymbol{z}_k)]^{-1} \mathbf{F}(\boldsymbol{x}_k, \boldsymbol{\mu}_k, \boldsymbol{z}_k).$$

The Jacobian is

$$\mathbf{J}(\boldsymbol{x}, \boldsymbol{\mu}, \boldsymbol{z}) = \begin{pmatrix} \mathbf{H} & \mathbf{A}^\top & \mathbf{0}_{n\times p} \\ \mathbf{A} & \mathbf{0}_{p\times p} & \mathbf{I}_p \\ \mathbf{0}_{p\times n} & \mathbf{Z} & \mathbf{D}_\mu \end{pmatrix}.$$

The Newton step solves

$$\begin{pmatrix} \mathbf{H} & \mathbf{A}^\top & \mathbf{0}_{n\times p} \\ \mathbf{A} & \mathbf{0}_{p\times p} & \mathbf{I}_p \\ \mathbf{0}_{p\times n} & \mathbf{Z} & \mathbf{D}_\mu \end{pmatrix} \begin{pmatrix} \Delta\boldsymbol{x} \\ \Delta\boldsymbol{\mu} \\ \Delta\boldsymbol{z} \end{pmatrix} = - \begin{pmatrix} \mathbf{H}\boldsymbol{x} + \mathbf{A}^\top \boldsymbol{\mu} - \boldsymbol{c} \\ \mathbf{A}\boldsymbol{x} + \boldsymbol{z} - \boldsymbol{d} \\ \mathbf{Z}\mathbf{D}_\mu \mathbf{1} \end{pmatrix} \equiv - \begin{pmatrix} \boldsymbol{r}_d \\ \boldsymbol{r}_p \\ \boldsymbol{r}_\mu \end{pmatrix},$$

where

$$\begin{pmatrix} \Delta\boldsymbol{x} \\ \Delta\boldsymbol{\mu} \\ \Delta\boldsymbol{z} \end{pmatrix} = \begin{pmatrix} \boldsymbol{x}_{k+1} \\ \boldsymbol{\mu}_{k+1} \\ \boldsymbol{z}_{k+1} \end{pmatrix} - \begin{pmatrix} \boldsymbol{x}_k \\ \boldsymbol{\mu}_k \\ \boldsymbol{z}_k \end{pmatrix}.$$

The complementarity equation $z_i \mu_i = 0$ is delicate, because we also need $z_i, \mu_i \geq 0$. This motivates the central path. The set of points $(\boldsymbol{x}^\tau, \boldsymbol{\mu}^\tau, \boldsymbol{z}^\tau)$ with $\boldsymbol{z}, \boldsymbol{\mu} > \mathbf{0}_p$ is called the **central path** if

$$\mathbf{F}(\boldsymbol{x}^\tau, \boldsymbol{\mu}^\tau, \boldsymbol{z}^\tau) = \begin{pmatrix} \mathbf{0}_n \\ \mathbf{0}_p \\ \tau \mathbf{1} \end{pmatrix}.$$

As $\tau \to 0$, the central path solution approaches a solution of the original system $\mathbf{F}(\boldsymbol{x}, \boldsymbol{\mu}, \boldsymbol{z}) = \mathbf{0}_{n+2p}$. A common choice is to set

$$\kappa = \frac{\boldsymbol{z}^\top \boldsymbol{\mu}}{p} \qquad \text{and} \qquad \tau = \sigma\kappa,$$

where $\sigma \in [0,1]$. The quantity κ is called the **complementarity measure**.

Lecture 28 — Friday, November 14, 2014

We are trying to deal with the condition $z_i, \mu_i \geq 0$. Using the slack variable $\boldsymbol{z} = \boldsymbol{d} - \mathbf{A}\boldsymbol{x}$, the conditions are:

$$\begin{aligned} \mathbf{H}\boldsymbol{x}^* + \mathbf{A}^\top \boldsymbol{\mu}^* - \boldsymbol{c} &= \mathbf{0}_n \\ \mathbf{A}\boldsymbol{x}^* + \boldsymbol{z} - \boldsymbol{d} &= \mathbf{0}_p \\ \mu_i z_i &= 0 \quad \text{for all } i \\ \mu_i, z_i &\geq 0 \end{aligned}$$

Define

$$\mathbf{F}(\boldsymbol{x}, \boldsymbol{\mu}, \boldsymbol{z}) = \begin{pmatrix} \mathbf{H}\boldsymbol{x} + \mathbf{A}^\top \boldsymbol{\mu} - \boldsymbol{c} \\ \mathbf{A}\boldsymbol{x} + \boldsymbol{z} - \boldsymbol{d} \\ \mathbf{Z}\mathbf{D}_\mu \mathbf{1} \end{pmatrix},$$

where $\mathbf{Z} = \operatorname{diag}(z_1, \ldots, z_p)$ and $\mathbf{D}_\mu = \operatorname{diag}(\mu_1, \ldots, \mu_p)$. Rather than solving $\mathbf{F}(\boldsymbol{x}, \boldsymbol{\mu}, \boldsymbol{z}) = \mathbf{0}_{n+2p}$ directly, we solve the central-path equations

$$\mathbf{F}(\boldsymbol{x}^\tau, \boldsymbol{\mu}^\tau, \boldsymbol{z}^\tau) = \begin{pmatrix} \mathbf{0}_n \\ \mathbf{0}_p \\ \tau \mathbf{1} \end{pmatrix}$$

for small $\tau > 0$.

Define

$$\bar{\mathbf{F}}(\boldsymbol{x}, \boldsymbol{\mu}, \boldsymbol{z}) = \mathbf{F}(\boldsymbol{x}, \boldsymbol{\mu}, \boldsymbol{z}) - \begin{pmatrix} \mathbf{0}_n \\ \mathbf{0}_p \\ \tau \mathbf{1} \end{pmatrix}.$$

Write

$$\bar{\mathbf{F}}(\boldsymbol{x}_k, \boldsymbol{\mu}_k, \boldsymbol{z}_k) = \begin{pmatrix} \boldsymbol{r}_d \\ \boldsymbol{r}_p \\ \boldsymbol{r}_\mu \end{pmatrix}.$$

To calculate the Newton step, solve

$$\begin{pmatrix} \mathbf{H} & \mathbf{A}^\top & \mathbf{0}_{n\times p} \\ \mathbf{A} & \mathbf{0}_{p\times p} & \mathbf{I}_p \\ \mathbf{0}_{p\times n} & \mathbf{Z} & \mathbf{D}_\mu \end{pmatrix} \begin{pmatrix} \Delta \boldsymbol{x} \\ \Delta \boldsymbol{\mu} \\ \Delta \boldsymbol{z} \end{pmatrix} = - \begin{pmatrix} \boldsymbol{r}_d \\ \boldsymbol{r}_p \\ \boldsymbol{r}_\mu \end{pmatrix}.$$

We determine τ using the complementarity measure $\kappa = \boldsymbol{z}^\top \boldsymbol{\mu}/p$, choosing $\tau = \sigma\kappa$ for some $\sigma \in [0,1]$. If $\mathbf{D}_\mu$ is invertible, then the Newton system can be reduced to

$$\begin{pmatrix} \mathbf{H} & \mathbf{A}^\top \\ \mathbf{A} & -\mathbf{D}_\mu^{-1}\mathbf{Z} \end{pmatrix} \begin{pmatrix} \Delta \boldsymbol{x} \\ \Delta \boldsymbol{\mu} \end{pmatrix} = \begin{pmatrix} -\boldsymbol{r}_d \\ -\boldsymbol{r}_p + \mathbf{D}_\mu^{-1}\boldsymbol{r}_\mu \end{pmatrix}.$$

This is a smaller system.

Box Constraints

Consider the **box-constrained** minimization problem

$$\min_{\boldsymbol{x}} \{ f(\boldsymbol{x}) \mid \boldsymbol{l} \le \boldsymbol{x} \le \boldsymbol{u} \}.$$

This is a very hard problem. A feasible point is **nondegenerate** if, for each index i,

$$(\nabla f(\boldsymbol{x}))_i = 0 \implies l_i < (\boldsymbol{x})_i < u_i.$$

Theorem 28.1 (Optimality conditions: box constraints)

1. (**First-order necessary condition**): if $\boldsymbol{x}^*$ is a local minimizer and $\boldsymbol{g}^* = \nabla f(\boldsymbol{x}^*)$, then
$$\begin{cases} (\boldsymbol{g}^*)_i = 0 & \text{if } l_i < (\boldsymbol{x}^*)_i < u_i \\ (\boldsymbol{g}^*)_i \le 0 & \text{if } (\boldsymbol{x}^*)_i = u_i \\ (\boldsymbol{g}^*)_i \ge 0 & \text{if } (\boldsymbol{x}^*)_i = l_i \end{cases} \qquad (*)$$

2. (**Second-order necessary condition**): let
$$\text{Free}(\boldsymbol{x}^*) = \{ i \mid l_i < (\boldsymbol{x}^*)_i < u_i \}.$$
If $\boldsymbol{x}^*$ is a local minimizer and $\boldsymbol{g}^* = \nabla f(\boldsymbol{x}^*)$, then the Hessian on the free variables, $\mathbf{H}^{\text{free}}$, is positive semidefinite.

3. (**Second-order sufficiency condition**): if $\boldsymbol{x}_*$ is nondegenerate and (*) holds and $\mathbf{H}^{\text{free}}$ is positive definite, then $\boldsymbol{x}_*$ is a local minimizer.

Example 28.2 Let

$$\mathbf{H} = \begin{pmatrix} -10 & 2 & 1 \\ 2 & 8 & 3 \\ 1 & 3 & 16 \end{pmatrix} \qquad \text{and} \qquad \text{Free} = \{2, 3\}.$$

If $x_1^* \in \{l_1, u_1\}$ while $x_2^* \in (l_2, u_2)$ and $x_3^* \in (l_3, u_3)$, then

$$\mathbf{H}^{\text{free}} = \begin{pmatrix} 8 & 3 \\ 3 & 16 \end{pmatrix}.$$

This free Hessian is positive definite.

Lecture 29 — Tuesday, November 18, 2014

Example 29.1 Let

$$\boldsymbol{l} = \begin{pmatrix} 1 \\ 1 \end{pmatrix}, \qquad \boldsymbol{u} = \begin{pmatrix} 2 \\ 2 \end{pmatrix}, \qquad \text{and} \qquad \boldsymbol{x}_* = \begin{pmatrix} 1 \\ 1.5 \end{pmatrix}.$$

If $\nabla f(\boldsymbol{x}_*) = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$, then $\boldsymbol{x}_*$ is nondegenerate: the only zero gradient component is the second one, and $(\boldsymbol{x}_*)_2 = 1.5$ is strictly inside its interval.

With the same bounds and $\boldsymbol{x}_*$, if $\nabla f(\boldsymbol{x}_*) = \begin{pmatrix} 0 \\ 0 \end{pmatrix}$, then $\boldsymbol{x}_*$ is degenerate, because the first gradient component is zero while $(\boldsymbol{x}_*)_1 = l_1$ is on the boundary.

The box-constrained minimization problem tends to be very difficult to solve. Two standard approaches are active-set methods and interior-point methods. In an **active-set approach**, set

$$\text{Free}(\boldsymbol{x}) = \{i \mid l_i < (\boldsymbol{x})_i < u_i\}.$$

Define also the active set

$$\mathrm{Act}(\boldsymbol{x}) = \{i \mid (\boldsymbol{x})_i = l_i \text{ or } (\boldsymbol{x})_i = u_i\}.$$

Starting from a feasible $\boldsymbol{x}_0$, a typical active-set iteration is:

1. At iterate $\boldsymbol{x}_k$, fix the active components and solve the reduced subproblem in the free variables.
2. Compute a feasible trial point with lower objective value.
3. If the first-order box-constrained optimality conditions are satisfied, stop.
4. Otherwise, modify the working active set (usually by releasing one active bound or adding a newly hit bound) and continue.

In a simplified setting, we seek $\boldsymbol{x}_{k+1}$ such that

$$f(\boldsymbol{x}_{k+1}) < f(\boldsymbol{x}_k) \qquad \text{and often} \qquad \mathrm{Free}(\boldsymbol{x}_k) \subseteq \mathrm{Free}(\boldsymbol{x}_{k+1}).$$

Example 29.2 For intuition, let $\boldsymbol{x} = (x_1, x_2, x_3)^\top$ represent age, height, and weight within box bounds. Suppose three feasible candidates are

$$\boldsymbol{x}^A = (35, 1.70, 180)^\top, \qquad \boldsymbol{x}^B = (30, 1.70, 170)^\top, \qquad \text{and} \qquad \boldsymbol{x}^C = (25, 1.60, 160)^\top.$$

If the sets of free indices at these points are

$$\mathrm{Free}(\boldsymbol{x}^A) = \{1, 3\}, \qquad \mathrm{Free}(\boldsymbol{x}^B) = \{3\}, \qquad \text{and} \qquad \mathrm{Free}(\boldsymbol{x}^C) = \{1, 2, 3\},$$

then the active-set method interprets this as follows: at $\boldsymbol{x}^B$ two variables are on bounds, so only one variable moves freely; after releasing active bounds we can move to a point such as $\boldsymbol{x}^C$ where all components are free. This is the mechanism by which the method updates working sets while continuing to decrease the objective.

In an **interior-point method**, each iteration stays strictly inside the box, with $\boldsymbol{l} < \boldsymbol{x}_k < \boldsymbol{u}$, and the iterates approach a solution $\boldsymbol{x}^*$. A typical trust-region subproblem has the form

$$\min_{\boldsymbol{d}} \left\{ \boldsymbol{d}^\top \nabla f(\boldsymbol{x}_k) + \frac{1}{2} \boldsymbol{d}^\top (\mathbf{H}_k + \mathbf{C}_k) \boldsymbol{d} \mid \|\mathbf{D}_k^{-1} \boldsymbol{d}\| \le \Delta_k \right\},$$

where $\mathbf{C}_k$ and $\mathbf{D}_k$ are diagonal matrices. Interior-point methods are very new and very exciting, and are being actively researched. Unfortunately they lie far beyond the understanding of a mere undergrad, so we will not pursue them further.

Actually, by 2014 interior-point methods were already quite mature. For problems with box constraints (which, contrary to these notes, are unusually *easy* to deal with), methods that exploit the constraints such as the L-BFGS-B algorithm are standard, and interior-point methods are usually overkill.

To solve a quadratic program

$$\min\left\{\boldsymbol{f}^\top\boldsymbol{x}+\frac{1}{2}\boldsymbol{x}^\top\mathbf{H}\boldsymbol{x}\mid \mathbf{A}\boldsymbol{x}\leq\boldsymbol{b},\ \mathbf{A}_{eq}\boldsymbol{x}=\boldsymbol{b}_{eq},\ \boldsymbol{l}\leq\boldsymbol{x}\leq\boldsymbol{u}\right\},$$

in MATLAB, use

```
[x, fval] = quadprog(H, f, A, b, Aeq, beq, l, u)
```

For the more general constrained nonlinear problem

$$\min\left\{h(\boldsymbol{x})\mid \mathbf{A}\boldsymbol{x}\leq\boldsymbol{b},\ \mathbf{A}_{eq}\boldsymbol{x}=\boldsymbol{b}_{eq},\ \boldsymbol{l}\leq\boldsymbol{x}\leq\boldsymbol{u}\right\},$$

use

```
[x, fval] = fmincon(fun, x0, A, b, Aeq, beq, l, u)
```

where x_0 is the initial value.

Penalty Functions

One **penalty function** approach replaces equality constraints by a penalty term:

$$\min_{\boldsymbol{x}}\{f(\boldsymbol{x})\mid \mathbf{A}\boldsymbol{x}=\boldsymbol{b},\ \boldsymbol{l}\leq\boldsymbol{x}\leq\boldsymbol{u}\}\rightsquigarrow\min\left\{f(\boldsymbol{x})+\frac{1}{2\mu}\|\mathbf{A}\boldsymbol{x}-\boldsymbol{b}\|^2\mid \boldsymbol{l}\leq\boldsymbol{x}\leq\boldsymbol{u}\right\}.$$

Taking μ small makes violations of $\mathbf{A}\boldsymbol{x}=\boldsymbol{b}$ expensive.

Lecture 30 — Thursday, November 20, 2014

Consider the box-constrained problem

$$\min_{\boldsymbol{x}}\{f(\boldsymbol{x}) \mid \mathbf{A}\boldsymbol{x} = \boldsymbol{b},\ \boldsymbol{l} \le \boldsymbol{x} \le \boldsymbol{u}\}.$$

A quadratic penalty replaces the equality constraint by a term in the objective:

$$\min_{\boldsymbol{x}} \left\{ p(\boldsymbol{x}) := f(\boldsymbol{x}) + \frac{1}{2\mu}\|\mathbf{A}\boldsymbol{x} - \boldsymbol{b}\|_2^2 \mid \boldsymbol{l} \le \boldsymbol{x} \le \boldsymbol{u} \right\}.$$

The Hessian of the penalized objective is

$$\nabla^2 p(\boldsymbol{x}) = \nabla^2 f(\boldsymbol{x}) + \frac{1}{\mu}\mathbf{A}^\top \mathbf{A}.$$

For small $\mu > 0$, violations of $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$ are expensive, so minimizers of the penalized problem are pushed toward feasibility. The same idea applies to nonlinear equality constraints,

$$\min_{\boldsymbol{x}}\{f(\boldsymbol{x}) \mid g(\boldsymbol{x}) = \boldsymbol{b},\ \boldsymbol{l} \le \boldsymbol{x} \le \boldsymbol{u}\},$$

by replacing $\|\mathbf{A}\boldsymbol{x} - \boldsymbol{b}\|_2^2$ with $\|g(\boldsymbol{x}) - \boldsymbol{b}\|_2^2$.

5. Portfolio Optimization Models

The Efficient Frontier

Let $S_1, \ldots, S_n$ be risky assets, let μ_i be the expected return on asset S_i, and let σ_{ij} be the covariance between the returns of S_i and S_j. Set

$$\boldsymbol{\mu} = (\mu_1, \ldots, \mu_n)^\top, \qquad \boldsymbol{\Sigma} = (\sigma_{ij})_{i,j=1}^n, \qquad \text{and} \qquad \mathbf{1} = (1, \ldots, 1)^\top.$$

The covariance matrix $\boldsymbol{\Sigma}$ is always symmetric positive semidefinite. For the closed-form derivations below, we assume it is positive definite. If x_i is the fraction of wealth invested in asset S_i, then the budget constraint is

$$\mathbf{1}^\top \boldsymbol{x} = 1.$$

The expected return and variance of the portfolio are

$$\mu_p = \boldsymbol{\mu}^\top \boldsymbol{x} \qquad \text{and} \qquad \sigma_p^2 = \boldsymbol{x}^\top \boldsymbol{\Sigma} \boldsymbol{x}.$$

Markowitz's model for efficient portfolios can be described in three equivalent ways. The **variance-efficient problem** fixes a target expected return and minimizes risk:

$$\min_{\boldsymbol{x}} \{\boldsymbol{x}^\top \boldsymbol{\Sigma} \boldsymbol{x} \mid \boldsymbol{\mu}^\top \boldsymbol{x} = \mu_p,\ \mathbf{1}^\top \boldsymbol{x} = 1\}.$$

The **expected-return-efficient problem** fixes a target risk and maximizes expected return:

$$\max_{\boldsymbol{x}} \{\boldsymbol{\mu}^\top \boldsymbol{x} \mid \boldsymbol{x}^\top \boldsymbol{\Sigma} \boldsymbol{x} = \sigma_p^2,\ \mathbf{1}^\top \boldsymbol{x} = 1\}.$$

The **parameter-efficient problem** (or **hybrid problem**) combines the two objectives:

$$\min_{\boldsymbol{x}} \left\{ -t\boldsymbol{\mu}^\top \boldsymbol{x} + \frac{1}{2} \boldsymbol{x}^\top \boldsymbol{\Sigma} \boldsymbol{x} \mid \mathbf{1}^\top \boldsymbol{x} = 1 \right\},$$

where $t \geq 0$ is fixed. When $t = 0$, only variance is minimized; as t increases, expected return receives more weight.

Assume $\boldsymbol{\mu}$ is not a multiple of $\mathbf{1}$. For the parameter-efficient model, the first-order condition gives

$$\boldsymbol{\Sigma} \boldsymbol{x} - t\boldsymbol{\mu} = \lambda \mathbf{1}.$$

Since $\boldsymbol{\Sigma}$ is nonsingular,

$$\boldsymbol{x} = t\boldsymbol{\Sigma}^{-1}\boldsymbol{\mu} + \lambda \boldsymbol{\Sigma}^{-1}\mathbf{1}.$$

Using $\mathbf{1}^\top \boldsymbol{x} = 1$,

$$\lambda = \frac{1 - t\mathbf{1}^\top \boldsymbol{\Sigma}^{-1}\boldsymbol{\mu}}{\mathbf{1}^\top \boldsymbol{\Sigma}^{-1}\mathbf{1}}.$$

Substituting this expression for λ gives an affine parametrization of the optimal portfolio:

$$\boldsymbol{x} = t\boldsymbol{\Sigma}^{-1}\boldsymbol{\mu} + \frac{1 - t\mathbf{1}^\top \boldsymbol{\Sigma}^{-1}\boldsymbol{\mu}}{\mathbf{1}^\top \boldsymbol{\Sigma}^{-1}\mathbf{1}} \boldsymbol{\Sigma}^{-1}\mathbf{1} = \underbrace{\frac{\boldsymbol{\Sigma}^{-1}\mathbf{1}}{\mathbf{1}^\top \boldsymbol{\Sigma}^{-1}\mathbf{1}}}_{=: \boldsymbol{h}_0} + \underbrace{\left(\boldsymbol{\Sigma}^{-1}\boldsymbol{\mu} - \frac{\mathbf{1}^\top \boldsymbol{\Sigma}^{-1}\boldsymbol{\mu}}{\mathbf{1}^\top \boldsymbol{\Sigma}^{-1}\mathbf{1}} \boldsymbol{\Sigma}^{-1}\mathbf{1} \right)}_{=: \boldsymbol{h}_1} t = \boldsymbol{h}_0 + \boldsymbol{h}_1 t.$$

Thus the expected return is

$$\mu_p = \boldsymbol{\mu}^\top \boldsymbol{x} = \boldsymbol{\mu}^\top \boldsymbol{h}_0 + t\boldsymbol{\mu}^\top \boldsymbol{h}_1 = \alpha_0 + \alpha_1 t,$$

where $\alpha_0 = \boldsymbol{\mu}^\top \boldsymbol{h}_0$ and $\alpha_1 = \boldsymbol{\mu}^\top \boldsymbol{h}_1$. The variance is

$$\sigma_p^2 = (\boldsymbol{h}_0 + \boldsymbol{h}_1 t)^\top \boldsymbol{\Sigma}(\boldsymbol{h}_0 + \boldsymbol{h}_1 t) = \boldsymbol{h}_0^\top \boldsymbol{\Sigma} \boldsymbol{h}_0 + 2t \boldsymbol{h}_1^\top \boldsymbol{\Sigma} \boldsymbol{h}_0 + t^2 \boldsymbol{h}_1^\top \boldsymbol{\Sigma} \boldsymbol{h}_1 = \beta_0 + 2\beta_1 t + \beta_2 t^2.$$

Here $\beta_0 = \boldsymbol{h}_0^\top \boldsymbol{\Sigma} \boldsymbol{h}_0$, $\beta_1 = \boldsymbol{h}_1^\top \boldsymbol{\Sigma} \boldsymbol{h}_0$, and $\beta_2 = \boldsymbol{h}_1^\top \boldsymbol{\Sigma} \boldsymbol{h}_1$. A direct calculation gives $\beta_1 = 0$ and $\beta_2 = \alpha_1$, so

$$\sigma_p^2 = \beta_0 + \alpha_1 t^2.$$

Since

$$t = \frac{\mu_p - \alpha_0}{\alpha_1} \qquad \text{and} \qquad t^2 = \frac{\sigma_p^2 - \beta_0}{\alpha_1},$$

the **efficient frontier** in mean–variance space is

$$\sigma_p^2 - \beta_0 = \frac{(\mu_p - \alpha_0)^2}{\alpha_1}.$$

The two branches of this parabola are shown in Figure 1.

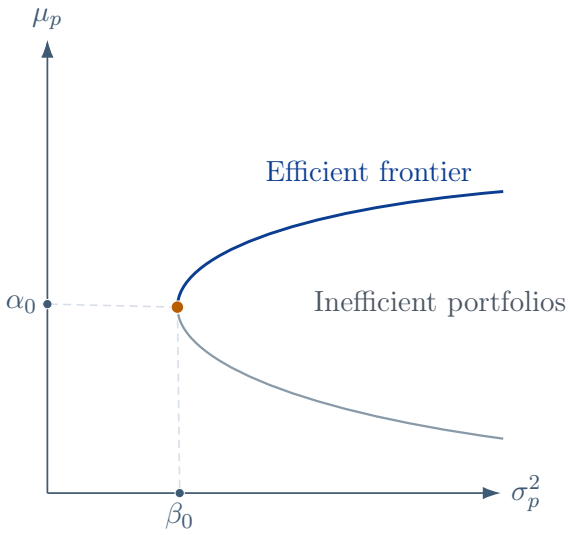

FIGURE 1

In Figure 1, the point (α_0, β_0) corresponds to the global minimum-variance portfolio. In practice, portfolio managers update their estimate of the minimum-variance portfolio every day. Those management fees aren't for nothing.

Lecture 31 — Friday, November 21, 2014

The same efficient frontier can be plotted in mean–standard deviation space. Since

$$\sigma_p = \sqrt{\beta_0 + \frac{(\mu_p - \alpha_0)^2}{\alpha_1}},$$

the upper branch may also be written as

$$\mu_p = \alpha_0 + \sqrt{\alpha_1(\sigma_p^2 - \beta_0)}.$$

Practical Portfolio Constraints

The basic Markowitz model is often extended by adding investment constraints. For example, Eva's investment problem has the form

$$\min_{\boldsymbol{x}} \left\{ -t\boldsymbol{\mu}^\top \boldsymbol{x} + \frac{1}{2}\boldsymbol{x}^\top \boldsymbol{\Sigma} \boldsymbol{x} \mid \mathbf{A}_1 \boldsymbol{x} = \boldsymbol{b}_1 \right\}.$$

Additional equality constraints give

$$\min_{\boldsymbol{x}} \left\{ -t\boldsymbol{\mu}^\top \boldsymbol{x} + \frac{1}{2}\boldsymbol{x}^\top \boldsymbol{\Sigma} \boldsymbol{x} \mid \mathbf{A}_1 \boldsymbol{x} = \boldsymbol{b}_1,\ \mathbf{A}_2 \boldsymbol{x} = \boldsymbol{d}_1 \right\}.$$

Closed-form solutions become less useful as these constraints become more detailed, and numerical quadratic programming methods are typically used.

Common practical constraints include the following:

1. **Box constraints:**

   $$\boldsymbol{l} \leq \boldsymbol{x} \leq \boldsymbol{u}.$$

   These enforce lower and upper bounds on each asset weight.

2. **Constraints on asset pairing:**

   $$K_{\min} \leq \sum_{i=1}^{n} \delta_i \leq K_{\max},$$

where

$$\delta_i = \begin{cases} 0 & x_i = 0, \\ 1 & x_i \neq 0. \end{cases}$$

These constraints bound the number of assets held. For instance, among five technology stocks, we might require holding at least two and at most three.

3. **Trade count constraints:**

$$T_{\min} \leq \sum_{i=1}^{n} \sigma_i \leq T_{\max},$$

where

$$\sigma_i = \begin{cases} 0 & x_i - (\boldsymbol{x}_0)_i = 0, \\ 1 & x_i - (\boldsymbol{x}_0)_i \neq 0. \end{cases}$$

These constraints bound the number of positions changed from an existing portfolio $\boldsymbol{x}_0$.

4. **Pair trading constraints:** For each asset index i, the variables $(\boldsymbol{x}_0)_i$ and x_i are treated as a paired state: $(\boldsymbol{x}_0)_i$ is the pre-trade holding and x_i is the post-trade holding. Constraints on these pairs can restrict allowed transitions (for example, no trade, increase, or decrease), which gives direct control over how each position is adjusted during rebalancing.

Capital Asset Pricing Model (CAPM)

Now suppose that, in addition to the n risky assets, there is one risk-free asset with return r. Let x_{n+1} be the fraction of wealth invested in the risk-free asset. Since the risk-free asset has zero variance and zero covariance with the risky assets,

$$\mu_p = \boldsymbol{\mu}^\top \boldsymbol{x} + r x_{n+1} \qquad \text{and} \qquad \sigma_p^2 = \begin{pmatrix} \boldsymbol{x} \\ x_{n+1} \end{pmatrix}^\top \begin{pmatrix} \boldsymbol{\Sigma} & \mathbf{0}_{n\times 1} \\ \mathbf{0}_{1\times n} & 0 \end{pmatrix} \begin{pmatrix} \boldsymbol{x} \\ x_{n+1} \end{pmatrix} = \boldsymbol{x}^\top \boldsymbol{\Sigma} \boldsymbol{x}.$$

The parameter-efficient problem becomes

$$\min_{\boldsymbol{x}, x_{n+1}} \left\{ -t(\boldsymbol{\mu}^\top \boldsymbol{x} + r x_{n+1}) + \frac{1}{2} \begin{pmatrix} \boldsymbol{x} \\ x_{n+1} \end{pmatrix}^\top \begin{pmatrix} \boldsymbol{\Sigma} & \mathbf{0}_{n\times 1} \\ \mathbf{0}_{1\times n} & 0 \end{pmatrix} \begin{pmatrix} \boldsymbol{x} \\ x_{n+1} \end{pmatrix} \mid \mathbf{1}^\top \boldsymbol{x} + x_{n+1} = 1 \right\}.$$

The first-order condition is

$$\begin{pmatrix} \boldsymbol{\Sigma}\boldsymbol{x} - t\boldsymbol{\mu} \\ -tr \end{pmatrix} = \lambda \begin{pmatrix} \mathbf{1} \\ 1 \end{pmatrix}.$$

Thus $\lambda = -tr$ and

$$\boldsymbol{\Sigma}\boldsymbol{x} = t(\boldsymbol{\mu} - r\mathbf{1}) \qquad \text{and} \qquad \boldsymbol{x} = t\boldsymbol{\Sigma}^{-1}(\boldsymbol{\mu} - r\mathbf{1}).$$

The budget constraint gives

$$x_{n+1} = 1 - \mathbf{1}^\top \boldsymbol{x} = 1 - t\mathbf{1}^\top \boldsymbol{\Sigma}^{-1}(\boldsymbol{\mu} - r\mathbf{1}).$$

Consequently,

$$\mu_p - r = t(\boldsymbol{\mu} - r\mathbf{1})^\top \boldsymbol{\Sigma}^{-1}(\boldsymbol{\mu} - r\mathbf{1}) \qquad \text{and} \qquad \sigma_p^2 = t^2(\boldsymbol{\mu} - r\mathbf{1})^\top \boldsymbol{\Sigma}^{-1}(\boldsymbol{\mu} - r\mathbf{1}).$$

Eliminating t yields the **capital market line**:

$$\mu_p - r = \sigma_p \left[(\boldsymbol{\mu} - r\mathbf{1})^\top \boldsymbol{\Sigma}^{-1}(\boldsymbol{\mu} - r\mathbf{1})\right]^{1/2}.$$

The **market portfolio** is the point on this line with no wealth in the risk-free asset. This occurs when

$$t_m = \frac{1}{\mathbf{1}^\top \boldsymbol{\Sigma}^{-1}(\boldsymbol{\mu} - r\mathbf{1})},$$

so that

$$\boldsymbol{x}_m = t_m \boldsymbol{\Sigma}^{-1}(\boldsymbol{\mu} - r\mathbf{1}) = \frac{\boldsymbol{\Sigma}^{-1}(\boldsymbol{\mu} - r\mathbf{1})}{\mathbf{1}^\top \boldsymbol{\Sigma}^{-1}(\boldsymbol{\mu} - r\mathbf{1})}.$$

Let $\mu_m = \boldsymbol{\mu}^\top \boldsymbol{x}_m$ and $\sigma_m^2 = \boldsymbol{x}_m^\top \boldsymbol{\Sigma} \boldsymbol{x}_m$. Since $\mathbf{1}^\top \boldsymbol{x}_m = 1$,

$$\sigma_m^2 = \boldsymbol{x}_m^\top \boldsymbol{\Sigma} \boldsymbol{x}_m = t_m(\boldsymbol{\mu} - r\mathbf{1})^\top \boldsymbol{x}_m = t_m(\mu_m - r).$$

It follows that the capital market line can be written as

$$\mu_p = r + \left(\frac{\mu_m - r}{\sigma_m}\right)\sigma_p.$$

Lecture 32 — Tuesday, November 25, 2014

The CAPM efficient frontier with one risk-free asset is described by

$$\min_{\boldsymbol{x}, x_{n+1}} \left\{ -t \begin{pmatrix} \boldsymbol{\mu}^\top & r \end{pmatrix} \begin{pmatrix} \boldsymbol{x} \\ x_{n+1} \end{pmatrix} + \frac{1}{2} \begin{pmatrix} \boldsymbol{x} \\ x_{n+1} \end{pmatrix}^\top \begin{pmatrix} \boldsymbol{\Sigma} & \mathbf{0}_{n\times 1} \\ \mathbf{0}_{1\times n} & 0 \end{pmatrix} \begin{pmatrix} \boldsymbol{x} \\ x_{n+1} \end{pmatrix} \mid \mathbf{1}^\top \boldsymbol{x} + x_{n+1} = 1 \right\}.$$

As derived previously,

$$\mu_p - r = \sigma_p \left[(\boldsymbol{\mu} - r\mathbf{1})^\top \boldsymbol{\Sigma}^{-1}(\boldsymbol{\mu} - r\mathbf{1})\right]^{1/2}$$

or, equivalently,

$$\mu_p = r + \left(\frac{\mu_m - r}{\sigma_m}\right)\sigma_p = r + \frac{\sigma_p}{\sigma_m}(\mu_m - r).$$

Figure 2 compares the capital market line with the efficient frontier for risky assets.

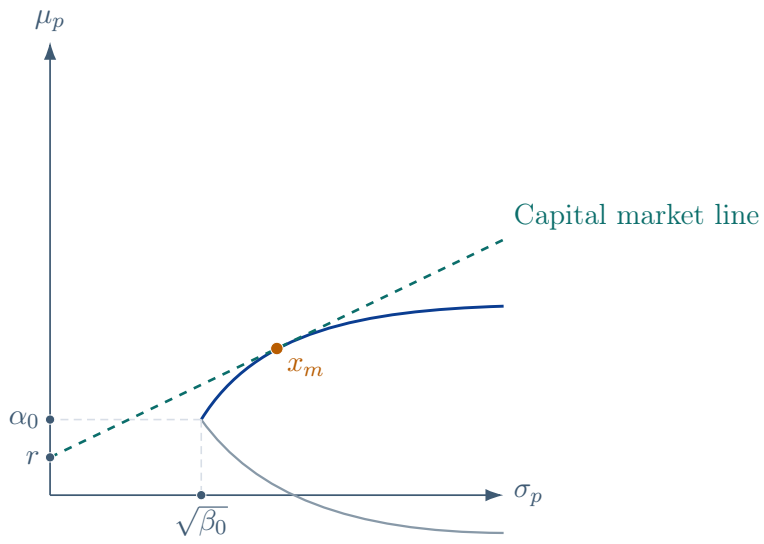

FIGURE 2

At the market portfolio x_m in Figure 2, we have $x_{n+1} = 0$. If short selling of the risk-free asset is prohibited, then $x_{n+1} \geq 0$, so the efficient frontier consists of two pieces: the capital market line from the risk-free asset to x_m, followed by the efficient frontier for risky assets from x_m onward. This combined frontier is continuous and differentiable at x_m.

Note that with a risk-free asset and unrestricted borrowing or lending at the risk-free rate, the efficient frontier in mean–standard deviation space is the capital market line. If borrowing is restricted by $x_{n+1} \geq 0$, then the line stops at the market portfolio.

In the form

$$\mu_p = r + \frac{\sigma_p}{\sigma_m}(\mu_m - r),$$

r is the risk-free return, $\mu_m - r$ is the **market premium**, and σ_p/σ_m is the portfolio's risk relative to the market portfolio. This ratio scales the market premium. For example, if the market return is 3%, the risk-free return is 1%, and $\sigma_p/\sigma_m = 1.5$, then

$$\mu_p = 1 + 1.5(3 - 1) = 4\%.$$

The Sharpe Ratio

The slope of the capital market line is the **Sharpe ratio**

$$R = \frac{\mu_p - r}{\sigma_p},$$

which measures excess expected return per unit of standard deviation. For the efficient frontier for risky assets,

$$\mu_p = \alpha_0 + \sqrt{\alpha_1(\sigma_p^2 - \beta_0)}.$$

Hence

$$\frac{\partial \mu_p}{\partial \sigma_p} = \frac{\alpha_1 \sigma_p}{\sqrt{\alpha_1(\sigma_p^2 - \beta_0)}}.$$

The tangent line to the frontier at the market-portfolio point (σ_m, μ_m) is

$$\mu_p = \mu_m + \frac{\alpha_1 \sigma_m}{\sqrt{\alpha_1(\sigma_m^2 - \beta_0)}}(\sigma_p - \sigma_m).$$

The risk-free return is the intercept of this tangent line at $\sigma_p = 0$. Since

$$\mu_m - \alpha_0 = \sqrt{\alpha_1(\sigma_m^2 - \beta_0)},$$

we obtain

$$r = \alpha_0 - \frac{\alpha_1 \beta_0}{\mu_m - \alpha_0}.$$

Conversely, given r,

$$\mu_m = \alpha_0 + \frac{\alpha_1 \beta_0}{\alpha_0 - r}.$$

Given the slope R of the capital market line, we can equivalently recover

$$\mu_m = \alpha_0 + \frac{\alpha_1 \sqrt{\beta_0}}{\sqrt{R^2 - \alpha_1}} \qquad \text{and} \qquad r = \alpha_0 - \sqrt{\beta_0(R^2 - \alpha_1)}.$$

Thus the risk-free return, the market portfolio, and the slope of the capital market line determine one another.

For a fixed risk-free return r, maximizing the Sharpe ratio means solving

$$\max_{\boldsymbol{x}} \left\{ \frac{\boldsymbol{\mu}^\top \boldsymbol{x} - r}{(\boldsymbol{x}^\top \boldsymbol{\Sigma} \boldsymbol{x})^{1/2}} \mid \mathbf{1}^\top \boldsymbol{x} = 1 \right\}.$$

Theorem 32.1 Maximizing the Sharpe ratio as above is equivalent to solving the Markowitz problem

$$\min_{\boldsymbol{x}} \left\{ -t\boldsymbol{\mu}^\top \boldsymbol{x} + \frac{1}{2}\boldsymbol{x}^\top \boldsymbol{\Sigma} \boldsymbol{x} \mid \mathbf{1}^\top \boldsymbol{x} = 1 \right\}.$$

Proof. Let

$$f(\boldsymbol{x}) = \frac{\boldsymbol{\mu}^\top \boldsymbol{x} - r}{(\boldsymbol{x}^\top \boldsymbol{\Sigma} \boldsymbol{x})^{1/2}}.$$

Then

$$\nabla f(\boldsymbol{x}) = \frac{(\boldsymbol{x}^\top \boldsymbol{\Sigma} \boldsymbol{x})^{1/2} \boldsymbol{\mu} - (\boldsymbol{\mu}^\top \boldsymbol{x} - r)(\boldsymbol{x}^\top \boldsymbol{\Sigma} \boldsymbol{x})^{-1/2} \boldsymbol{\Sigma} \boldsymbol{x}}{\boldsymbol{x}^\top \boldsymbol{\Sigma} \boldsymbol{x}}.$$

At an optimizer, $\nabla f(\boldsymbol{x}) = \lambda_1 \mathbf{1}$. Multiplying by $-(\boldsymbol{x}^\top \boldsymbol{\Sigma} \boldsymbol{x})^{3/2}/(\boldsymbol{\mu}^\top \boldsymbol{x} - r)$ gives

$$-\frac{\boldsymbol{x}^\top \boldsymbol{\Sigma} \boldsymbol{x}}{\boldsymbol{\mu}^\top \boldsymbol{x} - r} \boldsymbol{\mu} + \boldsymbol{\Sigma} \boldsymbol{x} = -\frac{(\boldsymbol{x}^\top \boldsymbol{\Sigma} \boldsymbol{x})^{3/2}}{\boldsymbol{\mu}^\top \boldsymbol{x} - r} \lambda_1 \mathbf{1}.$$

Equivalently,

$$t\boldsymbol{\mu} - \boldsymbol{\Sigma} \boldsymbol{x} = \lambda_2 \mathbf{1} \qquad \text{and} \qquad t = \frac{\boldsymbol{x}^\top \boldsymbol{\Sigma} \boldsymbol{x}}{\boldsymbol{\mu}^\top \boldsymbol{x} - r}.$$

This is exactly the first-order condition for the Markowitz problem

$$\min_{\boldsymbol{x}} \left\{ -t\boldsymbol{\mu}^\top \boldsymbol{x} + \frac{1}{2}\boldsymbol{x}^\top \boldsymbol{\Sigma} \boldsymbol{x} \mid \mathbf{1}^\top \boldsymbol{x} = 1 \right\}.$$

Thus maximizing the Sharpe ratio is equivalent, for the corresponding value of t, to solving the Markowitz problem.

Conversely, suppose the Sharpe ratio R is specified as the slope of a capital market line. The portfolio at tangency solves

$$\max_{\boldsymbol{x}} \left\{ \boldsymbol{\mu}^\top \boldsymbol{x} - R(\boldsymbol{x}^\top \boldsymbol{\Sigma} \boldsymbol{x})^{1/2} \mid \mathbf{1}^\top \boldsymbol{x} = 1 \right\}.$$

Its optimality condition is

$$\boldsymbol{\mu} - R\frac{\boldsymbol{\Sigma} \boldsymbol{x}}{(\boldsymbol{x}^\top \boldsymbol{\Sigma} \boldsymbol{x})^{1/2}} = \nu \mathbf{1},$$

or, after multiplying by $(\boldsymbol{x}^\top \boldsymbol{\Sigma} \boldsymbol{x})^{1/2}/R$,

$$\frac{(\boldsymbol{x}^\top \boldsymbol{\Sigma} \boldsymbol{x})^{1/2}}{R} \boldsymbol{\mu} - \boldsymbol{\Sigma} \boldsymbol{x} = \frac{(\boldsymbol{x}^\top \boldsymbol{\Sigma} \boldsymbol{x})^{1/2}}{R} \nu \mathbf{1}.$$

This again has the Markowitz form $t\boldsymbol{\mu} - \boldsymbol{\Sigma}\boldsymbol{x} = \lambda\mathbf{1}$, now with

$$t = \frac{(\boldsymbol{x}^\top \boldsymbol{\Sigma}\boldsymbol{x})^{1/2}}{R}.$$

Therefore a prescribed Sharpe ratio also selects a corresponding parameter-efficient portfolio. □

Editorial Note Lectures 33 and 34 were focused on a review of Assignment 2.

Appendix: Lecture and Tutorial Index